

UNIVERSIDAD MAYOR DE SAN ANDRÉS

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA ELECTRÓNICA



**SISTEMA DE VISIÓN ARTIFICIAL PARA
DETECTAR NEUMONÍA A PARTIR DE
IMÁGENES RADIOLÓGICAS UTILIZANDO
TÉCNICAS DE APRENDIZAJE PROFUNDO**

Proyecto de Grado presentado para optar al título de Licenciatura en Ingeniería
Electrónica

Postulante: Carlos Salvador Mendieta Villanueva

Tutor: Ing. Javier Sanabria García

La Paz - Bolivia

2025



**UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERIA**



LA FACULTAD DE INGENIERIA DE LA UNIVERSIDAD MAYOR DE SAN ANDRÉS AUTORIZA EL USO DE LA INFORMACIÓN CONTENIDA EN ESTE DOCUMENTO SI LOS PROPÓSITOS SON Estrictamente Académicos.

LICENCIA DE USO

El usuario está autorizado a:

- a) Visualizar el documento mediante el uso de un ordenador o dispositivo móvil.
- b) Copiar, almacenar o imprimir si ha de ser de uso exclusivamente personal y privado.
- c) Copiar textualmente parte(s) de su contenido mencionando la fuente y/o haciendo la cita o referencia correspondiente en apego a las normas de redacción e investigación.

El usuario no puede publicar, distribuir o realizar emisión o exhibición alguna de este material, sin la autorización correspondiente.

TODOS LOS DERECHOS RESERVADOS. EL USO NO AUTORIZADO DE LOS CONTENIDOS PUBLICADOS EN ESTE SITIO DERIVARA EN EL INICIO DE ACCIONES LEGALES CONTEMPLADAS EN LA LEY DE DERECHOS DE AUTOR.

Agradecimientos

A mis padres, Carlos y Natividad, por su apoyo constante e incondicional en cada proyecto que emprendo, incluido el que presento en este documento.

A los amigos y compañeros que tuve el placer de conocer a lo largo de la vida, y que, con su ayuda o consejo, fueron un pilar fundamental durante el desarrollo de este proyecto.

A los docentes que, a lo largo de la carrera, compartieron sus conocimientos y experiencias, inspirándonos a convertirnos en los profesionales que aspiramos ser, y que motivaron el nacimiento de esta iniciativa. En especial, al Ing. Javier Sanabria García, por su valiosa guía durante el proceso de redacción del presente trabajo.

A los mentores que la vida puso en mi camino, por sus reflexiones y enseñanzas que me ayudaron a definir mis objetivos y a tomar decisiones clave para culminar este proyecto.

A los médicos del sistema de salud boliviano, que día a día luchan por salvar vidas, especialmente en las zonas rurales, donde enfrentan grandes desafíos con recursos limitados. En particular, al licenciado en bioimagenología Miguel Ángel Quispe Maita, por su interés en esta propuesta y por colaborar activamente en la evaluación del prototipo.

A Dios, la Pachamama o cualquier ente espiritual que represente aquello que nos trasciende, por poner en mi camino a las personas adecuadas, cuidar de mi familia y brindarnos la salud necesaria para llegar hasta aquí.

RESUMEN

Este proyecto propone el desarrollo de un sistema de visión artificial basado en aprendizaje profundo para apoyar el diagnóstico de neumonía a partir de radiografías de tórax, con énfasis en la accesibilidad y en la contribución a un diagnóstico clínico preciso. La investigación parte del análisis del proceso diagnóstico actual en Bolivia, considerando la normativa vigente y la experiencia de profesionales en salud.

Se seleccionó la arquitectura ResNet-50, preentrenada con ImageNet, y se la adaptó para la clasificación binaria de neumonía. Se utilizaron los conjuntos de datos públicos Kermany y RSNA, y se desarrollaron algoritmos de preprocesamiento de imágenes para adaptar los datos a la arquitectura seleccionada. Con estos insumos, se entrenaron seis modelos, variando hiperparámetros y épocas de acuerdo con literatura especializada.

La evaluación se llevó a cabo mediante métricas como exactitud, precisión, sensibilidad, F1 y AUC, lo que permitió identificar el modelo con mejor rendimiento. El desarrollo se enmarcó en la metodología ágil Mobile-D, adecuada para aplicaciones móviles, lo cual facilitó la iteración continua entre diseño, implementación e integración.

El modelo final se integró en un prototipo funcional para Android que permite al usuario cargar o capturar una imagen radiológica y obtener una clasificación con su respectiva probabilidad, de manera offline. La validación, realizada junto a un radiólogo, arrojó resultados prometedores, confirmando el potencial del sistema como herramienta tecnológica de apoyo al diagnóstico médico en el sistema de salud boliviano.

ABSTRACT

This project proposes the development of a computer vision system based on deep learning to support pneumonia diagnosis from chest X-ray images, with an emphasis on accessibility and contributing to accurate clinical assessments. The research begins with an analysis of the current diagnostic process in Bolivia, taking into account existing regulations and the experience of healthcare professionals.

The ResNet-50 architecture, pre-trained with ImageNet, was selected and adapted for binary pneumonia classification. Public datasets from Kermany and RSNA were used, and image preprocessing algorithms were developed to adapt the data to the selected architecture. Using these inputs, six models were trained by varying hyperparameters and training epochs according to specialized literature.

The evaluation was carried out using metrics such as accuracy, precision, recall, F1-score, and AUC, which made it possible to identify the best-performing model. The development followed the agile Mobile-D methodology, suitable for mobile applications, which facilitated continuous iteration between design, implementation, and integration.

The final model was integrated into a functional Android prototype that allows users to upload or capture a radiographic image and obtain a classification with its associated probability, entirely offline. Validation, conducted in collaboration with a radiologist, yielded promising results, confirming the system's potential as a technological tool to support medical diagnosis within the Bolivian healthcare system.

ÍNDICE

1.	Capítulo I: Introducción	1
1.1.	Antecedentes	2
1.1.1.	Enfermedades pulmonares en Bolivia y el mundo	2
1.1.2.	Tecnologías en el diagnóstico de neumonía en Bolivia.....	4
1.2.	Situación actual	5
1.3.	Identificación del problema	10
1.4.	Objetivos	12
1.4.1.	Objetivo general.....	12
1.4.2.	Objetivos específicos	12
1.5.	Justificación	13
1.5.1.	Social.....	13
1.5.2.	Académica.....	13
1.5.3.	Tecnológica.....	14
1.6.	Alcances y limites	14
1.6.1.	Alcances	14
1.6.2.	Limites	16
2.	Capítulo II: Marco teórico.....	18
2.1.	Fundamentos de la Detección de Neumonía con Deep Learning	18
2.1.1.	Imágenes Médicas y Neumonía	18
2.1.1.1.	Introducción a la neumonía.....	18
2.1.1.1.1.	Clasificación	18
2.1.1.1.2.	Diagnóstico y evaluación.....	20
2.1.1.2.	Imágenes de rayos X en la detección de neumonía	22
2.1.1.2.1.	Vistas de radiografías de torax.....	23
2.1.1.3.	Terminología Radiológica para la Neumonía	23
2.1.1.4.	Sistemas de diagnóstico asistido por computadora (CAD).....	25
2.1.1.4.1.	Métodos tradicionales de procesamiento de imágenes	25
2.1.1.4.2.	Enfoques de aprendizaje automático (Machine Learning)	27
2.1.2.	Conjuntos de Datos	28
2.1.2.1.	Características de los conjuntos de datos.....	29
2.1.2.2.	DICOM	30
2.1.2.3.	Diseño de Estudio: Cohorte Retrospectiva	31
2.1.3.	Aprendizaje Profundo (Deep Learning).....	31

2.1.3.1.	Redes Neuronales Convolucionales (ConvNets).....	34
2.1.3.1.1.	Principios clave de las ConvNets.....	34
2.1.3.1.2.	Componentes de una ConvNet.....	35
2.1.3.1.3.	Aplicaciones de las ConvNets	36
2.1.3.2.	ResNet (Red Residual) y ResNet 50.....	37
2.1.3.2.1.	ResNet-50	38
2.1.3.2.1.	ResNet-50 en la Detección de Neumonía en Imágenes de Rayos X	39
2.1.3.3.	Transferencia de aprendizaje	41
2.1.3.4.	ImageNet.....	42
2.1.3.4.1.	Modelo ResNet-50 pre-entrenado con ImageNet	43
2.1.3.5.	Data augmentation	43
2.1.3.6.	Entrenamiento de Redes Neuronales Convolucionales	44
2.1.3.7.	Evaluación del modelo.....	47
2.1.3.7.1.	Métricas de evaluación	47
2.1.3.7.2.	Matriz de Confusión	48
2.1.3.7.3.	Curvas de aprendizaje (learning curves).....	48
2.2.	Herramientas de Desarrollo e Implementación.....	49
2.2.1.	Python	49
2.2.2.	PyTorch.....	49
2.2.2.1.	Herramientas de PyTorch	50
2.2.2.2.	Otras Librerías de Python	53
2.2.3.	Desarrollo de Aplicaciones Móviles con Android.....	54
2.2.3.1.	Introducción a Android.....	54
2.2.3.2.	Desarrollo con Android.....	55
2.2.3.3.	Android Studio.....	56
2.2.3.4.	Lenguaje de programación Java.....	56
2.2.3.5.	Bibliotecas de Android	57
2.3.	Metodología de Desarrollo: Mobile-D.....	58
2.3.1.	Fases de Mobile-D	59
3.	Capítulo III: Ingeniería de proyecto	60
3.1.	Introducción	60
3.2.	Exploración	60
3.2.1.	Establecimiento de partes interesadas.....	61
3.2.2.	Recopilación Inicial de Requisitos.....	61

3.2.2.1.	Análisis del Proceso Actual de Diagnóstico de Neumonía en Bolivia	61
3.2.2.1.1.	Análisis del sistema operativo Android	63
3.2.2.2.	Requerimientos del Sistema.....	64
3.2.2.2.1.	Requerimientos Funcionales	64
3.2.2.2.2.	Requerimientos No Funcionales	65
3.2.2.2.3.	Requerimientos Técnicos	66
3.2.2.2.4.	Requerimientos para el entrenamiento del modelo.....	66
3.2.3.	Establecimiento del Proyecto.....	67
3.2.3.1.	Elección de las Tecnologías.....	67
3.2.3.1.1.	Framework de Aprendizaje Profundo	68
3.2.3.1.2.	Tecnologías de Desarrollo de la Aplicación	68
3.2.3.1.3.	Estación de Trabajo.....	70
3.3.	Inicialización.....	71
3.3.1.	Configuración del Proyecto (Preparación y análisis de datos)	71
3.3.1.1.	Conjuntos de datos	72
3.3.1.2.	Preprocesamiento de Datos	81
3.3.2.	Planificación Inicial	84
3.3.2.1.	Arquitectura del Modelo (ResNet50).....	84
3.3.2.1.1.	Comparación de Modelos Preentrenados.....	84
3.3.2.1.2.	Selección de la Arquitectura	85
3.3.2.2.	Adaptación del Modelo.....	86
3.3.2.2.1.	Proceso de adaptación del modelo	86
3.3.2.2.2.	Diseño de la Nueva Capa de Clasificación	87
3.3.2.2.3.	Implementación Técnica	89
3.3.3.	Desarrollo del Modelo de Deep Learning.....	90
3.3.3.1.	Configuración del Entrenamiento	90
3.3.3.1.1.	Hiperparámetros Seleccionados.....	91
3.3.3.1.2.	Configuración de DataLoaders	92
3.3.3.2.	Proceso de Entrenamiento y Evaluación.....	94
3.3.3.2.1.	Implementación del Entrenamiento	94
3.3.3.2.2.	Etapas del Entrenamiento	96
3.3.3.3.	Guardado del Modelo	97
3.3.3.4.	Preparación para Implementación Móvil.....	98
3.4.	Producción	100

3.4.1.	Diseño de la Aplicación	101
3.4.1.1.	Consideraciones de UI/UX	102
3.4.1.2.	Elementos de la UI.....	102
3.4.1.3.	Flujo del Usuario.....	103
3.4.1.4.	Arquitectura de la Aplicación	104
3.4.2.	Implementación del código.....	106
3.4.2.1.	Estructura del Código	106
3.4.2.2.	Bibliotecas.....	108
3.4.2.3.	Funcionalidades Implementadas.....	109
3.4.2.4.	Manejo de Imágenes (Image Handling).....	109
3.4.2.5.	Integración de la Inferencia (Inference Integration)	110
3.4.2.6.	Visualización de Resultados (Result Display).....	111
3.5.	Fase de Estabilización.....	112
3.5.1.	Pruebas y Depuración	112
3.6.	Fase de Pruebas.....	112
3.6.1.	Evaluación del Modelo	112
3.6.2.	Pruebas de la Aplicación.....	113
4.	Capítulo IV: Análisis de resultados.....	114
4.1.	Comparación del desempeño de los modelos	114
4.1.1.	Descripción de los modelos	114
4.1.2.	Métricas de evaluación	115
4.1.2.1.	Conjunto de datos de prueba.....	116
4.1.2.2.	Procedimiento de evaluación	116
4.1.2.3.	Resultados de la evaluación con métricas.....	117
4.1.3.	Análisis de las curvas de aprendizaje.....	120
4.1.4.	Matrices de confusión	124
4.2.	Evaluación del prototipo por un radiólogo	126
4.2.1.	Metodología de evaluación	126
5.	Capítulo V: Conclusiones y recomendaciones.....	128
5.1.	Conclusiones	128
5.2.	Recomendaciones	133
6.	Bibliografía.....	135
7.	GLOSARIO DE TÉRMINOS.....	144
8.	ANEXOS.....	147

ANEXO A.1. Código Fuente del proyecto	147
ANEXO A.2. Output obtenido en la terminal al entrenar el modelo 6 (Con el conjunto de datos RSNA y 30 épocas)	161
ANEXO A.3. Interfaz de la aplicación desarrollada	162
ANEXO A.4. Resultados tabulados de la evaluación del especialista	163
ANEXO A.5. Tabla llenada por el especialista radiólogo.....	165
ANEXO A.6. Imágenes radiológicas del sistema de salud boliviana utilizadas para la evaluación por especialista.....	167
ANEXO A7. Transcripción de la entrevista realizada al Dr. Edgar Calderón	172
ANEXO A8. Arquitectura general del proceso de entrenamiento del modelo de aprendizaje profundo..	174

ÍNDICE DE FIGURAS

Figura 1.	Diagrama del proceso de diagnóstico de neumonía.....	11
Figura 2.	Clasificación general de la neumonía aguda.....	19
Figura 3.	Abordaje general de la neumonía adquirida en la comunidad en anfitriones normales	21
Figura 4.	Vista posteroanterior (PA) de una radiografía de tórax con algunos elementos que son tomados en cuenta el momento de evaluar la placa radiográfica	22
Figura 5.	Distorsión del espacio de entrada	33
Figura 6.	Propagación hacia adelante (forward pass).....	33
Figura 7.	Retropropagación (backward pass).....	34
Figura 8.	Por dentro de una red neuronal convolucional	37
Figura 9.	Conexión de salto (Aprendizaje residual).....	38
Figura 10.	Arquitectura ResNet-50.....	39
Figura 11.	Ejemplos de arquitecturas de redes neuronales para el set de datos ImageNet.....	40
Figura 12.	La transferencia de aprendizaje y el aprendizaje automático	41
Figura 13.	Tres maneras en las cuales el método de transferencia mejora el aprendizaje.....	41
Figura 14.	Ciclo de entrenamiento describiendo feedforward y backpropagation	45
Figura 15.	Resultado de la clase la aplicación de la clase torchvision.transforms.Resize.....	52
Figura 16.	Resultado de la aplicación de la clase torchvision.transforms.RandomHorizontalFlip	52
Figura 17.	Resultado de la aplicación de la clase torchvision.transforms.RandomVerticalFlip	53
Figura 18.	Resultado de la aplicación de la clase torchvision.transforms.RandomAffine	53
Figura 19.	Diagrama resumen de la ingeniería de proyecto	60
Figura 20.	Diagrama del proceso de diagnóstico de neumonía con la solución desarrollada.....	62
Figura 21.	Distribución de exámenes por edad y Clase en el conjunto de datos 2.....	75

Figura 22.	Algoritmo de conversión de imágenes DICOM a JPG	80
Figura 23.	Ejemplo de redimensionamiento de una imagen radiológica.....	81
Figura 24.	Ejemplo de volteo horizontal aleatorio.....	82
Figura 25.	Ejemplo de transformaciones afines aleatorias	82
Figura 26.	Rendimiento de clasificación de varias arquitecturas.....	85
Figura 27.	Diagrama del proceso de transferencia de aprendizaje aplicado.....	88
Figura 28.	Fragmento de código aplicando transferencia de aprendizaje.....	89
Figura 29.	Módulo de Preparación para el Entrenamiento	93
Figura 30.	Esquema inicial del entrenamiento del modelo.....	94
Figura 31.	Diagrama de flujo del proceso de entrenamiento	95
Figura 32.	Módulo de Post-Entrenamiento y Guardado del Modelo.....	99
Figura 33.	Arquitectura general del proceso de entrenamiento del modelo de aprendizaje profundo	100
Figura 34.	Vista de diseño del layout de la aplicación en Android Studio.....	101
Figura 35.	Flujo de Inferencia con Imagen Seleccionada.....	103
Figura 36.	Flujo de Inferencia con Imágenes de prueba.....	104
Figura 37.	Arquitectura de la aplicación móvil Android	105
Figura 38.	Diagrama de clases UML	108
Figura 39.	Interfaz de la aplicación una vez se ha realizado la inferencia.....	111
Figura 40.	Curvas ROC para los modelos entrenados con el conjunto de datos 1 (Modelo 1, 2 y 3)	119
Figura 41.	Curvas ROC para los modelos entrenados con el conjunto de datos 2 (Modelo 4, 5 y 6)	119
Figura 42.	Curvas de aprendizaje de los modelos 1, 2 y 3 que fueron entrenados con el conjunto de datos 1 (Kermany)	120
Figura 43.	Curvas de aprendizaje de los modelos 4, 5 y 6 que fueron entrenados con el conjunto de datos 2 (RSNA).....	121
Figura 44.	Curvas de aprendizaje de modelos representativos de cada conjunto de datos.....	122
Figura 45.	Curva de aprendizaje del modelo 6 (Entrenado durante 30 épocas) y con el conjunto de datos 2 (RSNA).....	123
Figura 46.	Matriz de confusión de los modelos 1, 2 y 3 que fueron entrenados con el conjunto de datos 1 (Kermany)	124
Figura 47.	Matriz de confusión de los modelos 4, 5 y 6 que fueron entrenados con el conjunto de datos 2 (RSNA).....	125

Figura 48.	Matriz de confusión producto de la validación de especialista	127
-------------------	---	-----

ÍNDICE DE TABLAS

Tabla 1.	Requerimientos funcionales del prototipo (aplicación móvil).....	64
Tabla 2.	Requerimientos no funcionales.....	65
Tabla 3.	Requerimientos técnicos	66
Tabla 4.	Requerimientos del conjunto de datos y entrenamiento del modelo	66
Tabla 5.	Especificaciones estación de trabajo.....	70
Tabla 6.	Resumen del conjunto de datos 1.....	72
Tabla 7.	Distribución de edades por rangos.....	74
Tabla 8.	Muestra de los primeros 5 registros del archivo stage_2_detailed_class_info.csv.....	76
Tabla 9.	Muestra de los primeros 5 registros del archivo stage_2_train_labels.csv	78
Tabla 10.	Resumen stage_2_detailed_class_info.csv	78
Tabla 11.	Resumen stage_2_train_labels.csv	79
Tabla 12.	Resumen del conjunto de datos 2.....	79
Tabla 13.	Comparación de arquitecturas.....	84
Tabla 14.	Resumen del proceso de adaptación de modelo	86
Tabla 15.	Modificaciones en la arquitectura de ResNet-50	89
Tabla 16.	Comparación de diferentes tasas de aprendizaje y tamaños de lote	90
Tabla 17.	Configuración de DataLoaders	92
Tabla 18.	Etapas del Entrenamiento	96
Tabla 19.	Consideraciones de UI/UX	102
Tabla 20.	Hiperparámetros utilizados para el entrenamiento de los diferentes modelos.....	114
Tabla 21.	Descripción inicial de los modelos	115
Tabla 22.	Resumen del conjunto de datos para realizar la evaluación	116
Tabla 23.	Resumen de evaluación de los 6 modelos.....	117

1. Capítulo I: Introducción

Desde finales del año 2019 y hasta mediados del año 2022 el mundo se vio envuelto en una crisis social, económica y, fundamentalmente sanitaria, debido a la detección del virus COVID-19, a causa de esta enfermedad se pudo evidenciar que todo el sistema de salud de Bolivia y de otros países de la región presentaban muchas deficiencias en la mayoría de sus áreas. Una de ellas es la neumología, especialidad de la medicina que se ocupa del estudio, diagnóstico, tratamiento y prevención de las enfermedades que afectan los pulmones y las vías respiratorias. El impacto fue significativo, ya que una de las principales consecuencias del COVID-19 era el deterioro de los pulmones, lo que en los casos más críticos derivaba en insuficiencia respiratoria y, en muchos casos, en la muerte del paciente.

Pero estaríamos equivocados al pensar que solo con esta pandemia la neumología se había convertido en un área que requería atención. Solo hace falta observar que para el 2015 la Organización Mundial de la Salud (OMS) reportaba que uno de los tipos más comunes de Infecciones Respiratorias Agudas (IRAS) era la neumonía, responsable del 15% de todas las defunciones en menores de 5 años, lo que equivale aproximadamente a unas 920 136 muertes infantiles ese año, observándose una mayor incidencia de éstas estadísticas en países de bajos y medianos ingresos y que tanto las IRAS como las enfermedades respiratorias crónicas representan una de las principales causas de uso de los servicios de salud en los países en vías de desarrollo [1].

Observando estos datos, que probablemente se incrementaron durante la pandemia, es casi inevitable buscar alternativas que intenten solucionar de una u otra manera problemáticas referidas a la neumología en nuestro país.

Es en ese sentido que el presente proyecto propone desarrollar un sistema que, mediante el entrenamiento de una red neuronal convolucional con imágenes radiológicas etiquetadas, pueda detectar la presencia de patologías específicas en los pulmones, como lo es la neumonía, otorgando una alternativa de consulta a personal médico el momento de evaluar la existencia de la enfermedad en una placa radiográfica. Además, para las posteriores pruebas por parte de médicos, radiólogos o especialistas, el sistema contempla la implementación de una solución informática como es una aplicación móvil Android mediante la cual se pueda acceder de una manera sencilla e intuitiva a esta herramienta.

Se espera que esta herramienta sea probada por personal médico, abriendo la posibilidad en base a las conclusiones que se obtengan, de que en un futuro este sistema pueda ser utilizado también por médicos que no tengan la especialidad o que se encuentren en la etapa de especialización, intentando de esta manera buscar soluciones a la falta de especialistas debido a la alta demanda de pacientes en emergencias como la que ocurrió a lo largo del año 2020 y durante las olas de contagio de COVID-19 posteriores.

1.1. Antecedentes

1.1.1. Enfermedades pulmonares en Bolivia y el mundo

Enfermedades pulmonares como la Enfermedad Pulmonar Obstructiva Crónica (EPOC), el asma y las Infecciones Respiratorias Agudas (IRA) se encuentran entre las causas más comunes de muertes relacionadas con salud en el mundo [2]. En países de ingresos bajos, o medianos bajos como Bolivia [3][4], las estadísticas varían colocando a las infecciones de las vías respiratorias inferiores en un segundo lugar como causa de defunción. Uno de los tipos de infecciones respiratorias agudas más comunes es la neumonía, misma que es responsable del 15% de todas las defunciones de menores de 5 años y se calcula que mató a unos 740 180 niños en 2019 [5].

La neumonía es una enfermedad común causada por diferentes especies microbianas como bacterias, virus y hongos, esta infección provoca la inflamación de uno o ambos parénquimas pulmonares. Sin embargo, otras causas de neumonía incluyen la aspiración de alimentos y la exposición a productos químicos. Según la infección, la neumonía se produce como resultado de la inflamación causada por patógenos que hacen que los alvéolos de los pulmones se llenen de líquido o pus y, por lo tanto, provocan una disminución del intercambio de dióxido de carbono (CO₂) y oxígeno (O₂) entre la sangre y los pulmones, lo que hace que a las personas infectadas les cueste respirar. [6]

En Bolivia según reportes semanales de vigilancia epidemiológica del Ministerio de Salud y Deportes, durante el 2023 se habrían reportado 145 664 casos de neumonía y 117 muertes de menores de 5 años por esa infección a lo largo de todo el territorio (SNIS-VE, 2024) [7].

Se observa entonces que tanto la neumonía como otras enfermedades pulmonares suponen un problema mundial para el cual constantemente se realizan campañas y planes que intentan reducir tanto muertes y contagios, entre ellos, por ejemplo, encontramos al Plan de Acción Global Integrado para la Neumonía y la Diarrea (GAPPD).

El GAPPD lanzado por la OMS y UNICEF en 2013, a partir del Objetivo de Desarrollo del Milenio (ODM) 4 (Reducir la mortalidad infantil) y el ahora Objetivo de Desarrollo Sostenible (ODS) 3 (Salud y Bienestar), para reducir la carga masiva de muertes por neumonía y diarrea entre los niños. Este fue el primer marco global para establecer objetivos nacionales de mortalidad por neumonía y diarrea. Para 2025, por ejemplo, uno de sus objetivos es que todos los países deberían haber reducido las muertes por neumonía infantil a menos de tres por cada 1000 bebés nacidos, otros objetivos también fueron establecidos para servicios de:

- Prevención

- Diagnóstico
- Tratamiento

Incluyendo una cobertura del 90% de las vacunas contra la neumonía, 90% acceso a tratamientos recomendados entre otros [8]

Como pudimos observar, este y otros planes sitúan al diagnóstico de la enfermedad como un aspecto importante. En Bolivia según las Normas Nacionales de Atención Clínica (Ministerio de Salud y Deportes, Bolivia 2012) [9] para enfermedades relacionadas con neumonía como:

- Neumonía adquirida en la comunidad (NAC) en niños
- NAC en adultos
- Neumonía grave, neumonía muy grave
- Otras neumonías (neumonía intra-hospitalaria (NIH))

Se indica reiteradamente la importancia de la radiografía de tórax para el diagnóstico de la infección, estableciéndose, por ejemplo: como un criterio de gravedad (un resultado de rayos X donde se muestre afectación multilobar, cavitación o derrame pleural); como un examen complementario de gabinete o también como un criterio de alta (una imagen radiológica no complicada)

1.1.2. Tecnologías en el diagnóstico de neumonía en Bolivia

En cuanto al diagnóstico por imagenología en Bolivia, a partir de la pandemia causada por el virus SARS-CoV-2 causante de la enfermedad por coronavirus del 2019 (COVID-19), varios entes hospitalarios en La Paz buscaron mejorar sus métodos de diagnóstico, entre ellos:

- El Hospital del Norte (Establecimiento de salud de tercer nivel situado en la ciudad de El Alto) en 2020 reportó haber implementado tecnología de inteligencia artificial para ver en tiempo real los estudios tomográficos y radiográficos de lóbulos

pulmonares de pacientes con COVID-19, en el reporte también describieron que se trabajó con tecnología Cloud de Huawei y con ello se podría atender diariamente de 50 a 100 pacientes afectados por el coronavirus [10]

- El Hospital Arco Iris (Establecimiento de salud de segundo nivel situado en la ciudad de La Paz) en 2021 reportó haber adquirido un tomógrafo que incorpora software con inteligencia artificial para precisar el diagnóstico de enfermedades cardiovasculares, pulmonares, neurovasculares y de COVID 19 entre otros [11].

1.2. Situación actual

La neumonía se suele detectar inicialmente a partir de un examen físico evaluando distintos sonidos captados por un estetoscopio que utiliza el médico, por ejemplo, para analizar la inhalación y exhalación del paciente. Además, para dar un diagnóstico más preciso se suelen realizar radiografías de tórax, tomografías computarizadas de tórax, pruebas de sangre y otros. En la radiografía de tórax, se obtienen imágenes de la estructura interna del tórax enfocadas principalmente en el área de los pulmones. [12]

En una búsqueda de literatura relacionada al estado del arte en detección de enfermedades, encontramos que el aprendizaje profundo (Deep Learning) está teniendo avances muy importantes en el área de la salud.

El aprendizaje profundo representa la última iteración en una progresión de tecnologías de inteligencia artificial que han permitido que las máquinas imiten la inteligencia humana de maneras cada vez más sofisticadas e independientes. Los primeros sistemas médicos de inteligencia artificial dependían en gran medida de expertos para entrenar computadoras mediante la codificación del conocimiento clínico como reglas lógicas para escenarios clínicos específicos. Los sistemas de aprendizaje automático más avanzados se entrenan para aprender estas reglas al

identificar y sopesar características relevantes de los datos, como píxeles de imágenes médicas o información sin procesar de registros de salud electrónicos. [13]

Es así que para la detección de enfermedades mediante aprendizaje profundo como la neumonía, la retinopatía diabética, el cáncer de piel y otras, se puede encontrar un incremento de investigaciones publicadas en la última década, ello muy relacionado al incremento de sets de datos (imágenes etiquetadas por especialistas, por ejemplo) y al incremento y accesibilidad computacional por parte de los investigadores (mejoras y reducción de costes en cuanto a memoria, procesamiento y hardware en general se refiere).

Referido a detección de neumonía mediante Deep Learning encontramos trabajos a partir del año 2016, mejorando métodos y arquitecturas de redes neuronales profundas, año tras año.

A continuación, se presentan dos trabajos presentados en 2021 y uno presentado en 2022 que entre otros más serán referencias principales en el presente proyecto

- **Pneumonia detection in chest X-ray images using an ensemble of deep learning models**

Estudio presentado en la revista científica PLOS ONE en septiembre de 2021 por Rohit Kundu, Ritacheta Das, Zong Woo Geem, Gi-Tae Han y Ram Sarkar, investigadores de la Universidad de Jadavpur (Kolkata, India) y la Universidad de Gachon (Seongnam, South Korea), en este trabajo proponen usar un arreglo de 3 arquitecturas donde se ponderaría los resultados de las mismas según las métricas obtenidas y de esta manera se pueda encontrar una predicción final más precisa sobre la presencia o no de neumonía en una imagen radiográfica. Además, en el artículo indican que los resultados de su estudio fueron superiores a los de los métodos ‘estado del arte’ y que su método

funcionó mejor que las técnicas de conjunto ampliamente utilizadas en la actualidad.
[14]

- **CGNet: A graph-knowledge embedded convolutional neural network for detection of pneumonia**

Estudio presentado en ‘Information Processing and Management Volume 58, Issue 1’ revista publicada en enero de 2021 mediante Elsevier por Xiang Yu, Shui-Hua Wang y Yu-Dong Zhang investigadores de la Universidad de Leicester (Leicester, UK) y la Universidad King Abdulaziz (Jeddah, Arabia Saudita). En este estudio los investigadores proponen otro tipo de metodología para la detección de neumonía mediante el análisis de imágenes radiográficas, a diferencia del anterior estudio proponen una arquitectura nueva, utilizando más y diferentes etapas esperando obtener resultados más precisos, además añaden un análisis de sus resultados con datos relacionados a neumonía causada por COVID-19 mediante rayos X y tomografía computarizada. [15]

- **An Adaptive and Altruistic PSO-based Deep Feature Selection Method for Pneumonia Detection from Chest X-Rays**

Estudio presentado en la revista científica Applied Soft Computing en agosto del año 2022 por Rishav Pramanik, Sourodip Sarkar y Ram Sarkar, investigadores del Departamento de Ciencias de la Computación e Ingeniería de la Universidad de Jadavpur (Kolkata, India) y del Departamento de Ingeniería Electrónica y de la Comunicación del Heritage Institute of Technology (Kolkata, India). En este trabajo, proponen un sistema de ayuda al diagnóstico por ordenador (CAD) para la detección

de la neumonía a partir de radiografías de tórax, utilizando los conceptos de aprendizaje profundo y un algoritmo metaheurístico. [34]

Adicionalmente en una revisión de trabajos a nivel local, pudimos encontrar que en la Universidad Mayor de San Andrés se desarrollaron proyectos que involucran algún subconjunto de la inteligencia artificial, proyectos que se enfocaron en ayudar en la detección de algún tipo de enfermedad y algunos que combinaron esas dos características utilizando conceptos de inteligencia artificial para la detección de enfermedades, a continuación citaremos algunos de ellos mismos que también serán utilizados como referencia a lo largo de este proyecto:

- **Clasificación de arritmias cardiacas utilizando análisis de componentes principales y redes neuronales pre alimentadas en segmentos cortos de electrocardiograma**

Proyecto presentado en la carrera de Ingeniería Electrónica, en 2019 por Yamil Marzana, el mismo presenta el diseño y desarrollo de un sistema que permite la clasificación de arritmias cardiacas, se tomará en cuenta la metodología para el análisis de un proyecto que implementa redes neuronales relacionado con una problemática médica [16]

- **Aprendizaje fin a fin para la conducción autónoma de vehículos domésticos usando visión artificial y redes neuronales convolucionales**

Proyecto presentado en la carrera de Ingeniería Electrónica, en 2019 por Jose Laruta, en este proyecto el autor realiza el análisis de requerimientos y el diseño de varios subsistemas para la conducción autónoma, entre ellos un subsistema de adquisición de datos y entrenamiento, un subsistema de inferencia y control automático basado en el uso de redes neuronales y otros, además realiza un análisis de los resultados de

entrenamiento e implementación. Los métodos usados para afrontar las anteriores tareas también serán tomadas en cuenta en el presente proyecto [17]

- **SISTEMA DE CONTEO DE LEUCOCITOS EN MUESTRAS DE SANGRE HUMANA POR RECuento DIFERENCIAL AUTOMATIZADO – Caso de Estudio: Aplicación de técnicas de procesamiento a imágenes capturadas por microscopios del Laboratorios de Análisis Clínico del Hospital De Clínicas, La Paz – Bolivia**

Proyecto presentado en la carrera de Ingeniería Electrónica, en 2018 por Jimmy Linares, en este proyecto se propone el diseño y desarrollo de un sistema que permite automatizar el conteo de leucocitos, el autor resalta el reto que presento el diseño y la posterior validación de un sistema de clasificación y el procesamiento de imágenes, aspectos que serán tomados en cuenta en el presente proyecto [18]

- **IDENTIFICACIÓN DE ANOMALÍAS COVID-19 EN RADIOGRAFÍAS DE TÓRAX MEDIANTE REDES NEURONALES Y EL SISTEMA RALE**

Este proyecto fue desarrollado en la Universidad Mayor de San Andrés y presentado en la carrera de Informática de la Facultad de Ciencias Puras y Naturales en 2022 por Jaime Antonio Montecinos Marquez. Este trabajo propone un sistema para la detección de neumonía atípica asociada al SARS-CoV-2 (COVID-19) mediante el uso de redes neuronales convolucionales (CNN) y el sistema RALE (Evaluación Radiográfica del Edema Pulmonar). El modelo emplea redes neuronales convolucionales para clasificar y extraer características de radiografías de tórax, integrando el sistema de puntuación RALE, valorando extensión y opacidad pulmonar [72].

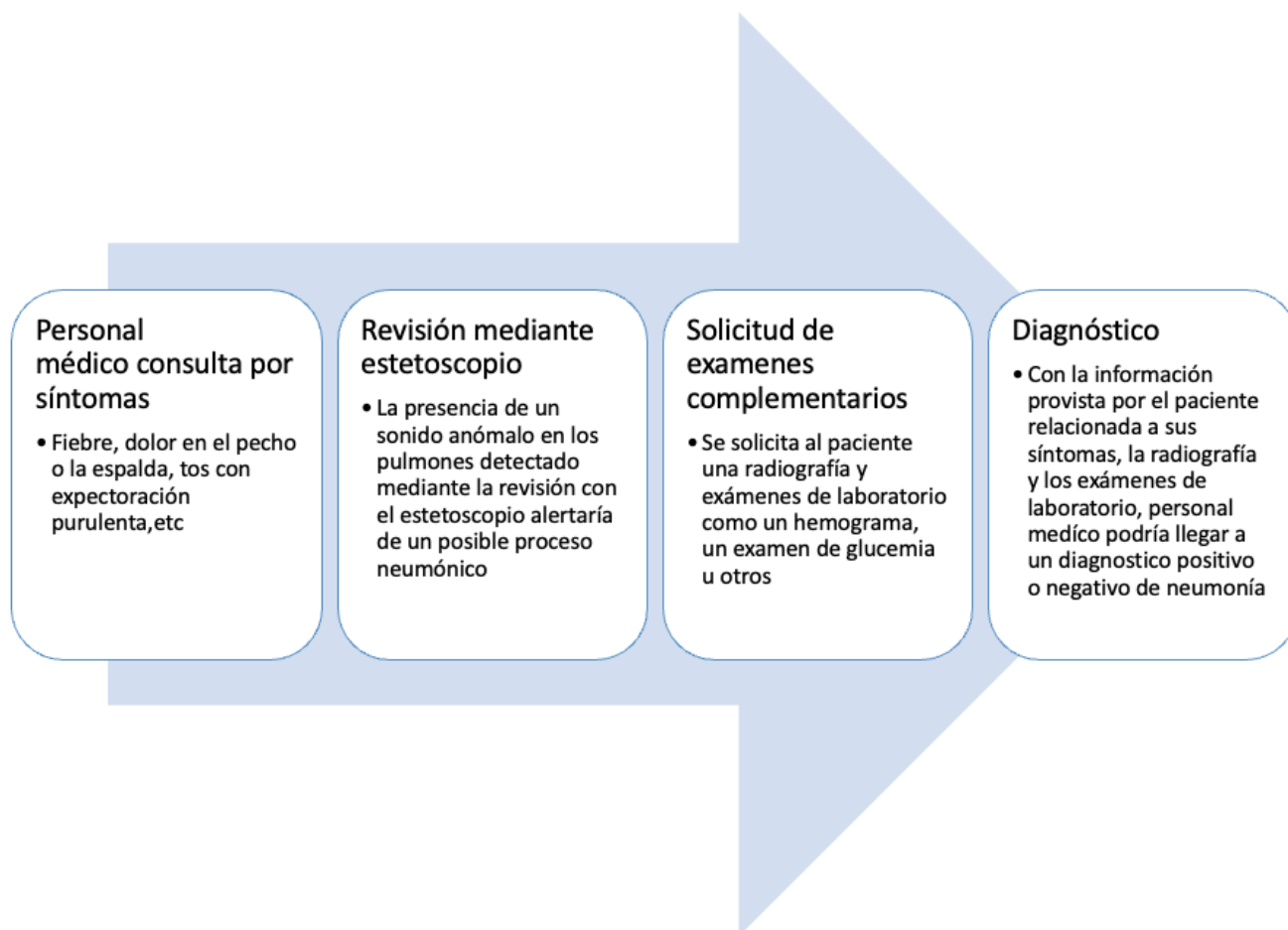
1.3. Identificación del problema

La neumonía es una de las principales causas de mortalidad a nivel mundial y nacional, como se detalló en los antecedentes del presente proyecto (Véase Sección 1.1). Organizaciones como la OMS han señalado que el diagnóstico temprano, junto con un tratamiento adecuado, es un factor crítico para reducir las complicaciones que pueden conducir a la muerte [19]. Este diagnóstico temprano representa un desafío en el contexto boliviano, no solo debido a la complejidad intrínseca de la enfermedad, sino también por las limitaciones en los recursos disponibles.

Actualmente, el diagnóstico de neumonía puede realizarse mediante diversas técnicas, entre ellas radiografías de tórax, tomografías computarizadas, pruebas de esputo, análisis de gases en sangre, broncoscopias, entre otras [6]. Sin embargo, las radiografías de tórax son el método clínico más común y accesible para este propósito [20]. Verificando ello en el contexto nacional boliviano mediante una entrevista realizada al Dr. Edgar Calderón, neumólogo del Instituto Nacional de Tórax (Establecimiento de salud de tercer nivel situado en la ciudad de La Paz), donde describió el proceso para realizar el diagnóstico de neumonía (Véase Figura 1), situando a la radiografía como un examen complementario habitual en el diagnóstico de la enfermedad.

A pesar de ello, interpretar estas imágenes representa una tarea compleja incluso para médicos especialistas y radiólogos experimentados, ya que las características de la neumonía en las radiografías pueden confundirse con otras enfermedades o anomalías benignas [21]. Esta dificultad se agrava por la variabilidad entre especialistas, los diferentes agentes causales (bacterias, virus, hongos, etc.) y las posibles inconsistencias en los diagnósticos, que pueden derivar en falsos positivos (tratamientos innecesarios) o falsos negativos (agravamiento de la enfermedad) [22].

Figura 1. Diagrama del proceso de diagnóstico de neumonía



Nota. Adaptado de una entrevista a Dr. Edgar Calderón (neumólogo del Instituto Nacional de Tórax de La Paz) en fecha 7 de septiembre de 2022

El problema se intensifica en Bolivia debido a factores estructurales. Según datos del Banco Mundial, en 2017 el país contaba con solo 1.03 médicos por cada 1,000 habitantes, un índice que apenas alcanza el mínimo recomendado por la OMS [23]. Esta situación es aún más crítica en áreas rurales, donde la falta de especialistas, servicios médicos especializados y la lejanía de los centros de atención complican el acceso oportuno al diagnóstico. Además, los costos elevados de estudios complementarios, como una tomografía de tórax en instituciones privadas que oscilaban alrededor de 650 Bs para el 2021 [11], limitan aún más las posibilidades de detección temprana.

Por lo tanto, surge como problemática la ausencia de soluciones tecnológicas accesibles y efectivas que puedan integrarse al proceso de diagnóstico clínico de la neumonía, especialmente en entornos de bajos recursos como los de Bolivia. Este vacío plantea la necesidad de explorar alternativas que permitan aprovechar los avances en tecnología para reducir las desigualdades en la calidad de atención médica y mejorar la capacidad de diagnóstico del personal sanitario.

1.4. Objetivos

1.4.1. Objetivo general

Desarrollar un sistema de visión artificial basado en técnicas de aprendizaje profundo para la detección de neumonía en imágenes radiológicas, que proporcione a los profesionales de la salud una solución tecnológica accesible y que contribuya a un diagnóstico clínico preciso.

1.4.2. Objetivos específicos

Para alcanzar el objetivo general será necesario:

- Analizar el proceso actual de diagnóstico de neumonía en Bolivia a partir del estudio de la normativa vigente y la experiencia del personal de salud involucrado en la interpretación de radiografías de tórax, permitiendo identificar limitaciones y oportunidades de mejora que orienten el diseño del sistema de visión artificial propuesto.
- Investigar y seleccionar arquitecturas de redes neuronales convolucionales y métodos de aprendizaje profundo adecuados para garantizar que el modelo desarrollado ofrezca un alto nivel de precisión en la clasificación de imágenes radiológicas.
- Identificar y procesar conjuntos de datos relevantes de imágenes radiológicas etiquetadas con el fin de conformar el conjunto de entrenamiento y validación

requerido para desarrollar y evaluar el modelo, fomentando así su capacidad de generalización.

- Desarrollar y entrenar un modelo de aprendizaje profundo para la detección de neumonía seleccionando los hiperparámetros más adecuados que permitan obtener métricas de evaluación precisas y confiables.
- Desarrollar un prototipo funcional que integre el modelo entrenado y facilite su uso por parte del personal médico permitiendo validar su desempeño en un entorno clínico.
- Validar el sistema desarrollado mediante pruebas con personal médico especializado para medir su precisión y comparar los resultados obtenidos con métodos tradicionales de diagnóstico.

1.5. Justificación

1.5.1. Social

El presente proyecto propone una alternativa de consulta a personal médico en el momento de evaluar la existencia de neumonía en una placa de rayos X de tórax, de esta manera se espera se pueda reducir el número de resultados erróneos (falsos positivos o falsos negativos). Esto contribuirá a una identificación más temprana de la enfermedad y, en consecuencia, a un inicio oportuno del tratamiento. Además, se espera que esta herramienta facilite el acceso a diagnósticos confiables en zonas con recursos limitados, beneficiando a la población general y fortaleciendo el sistema de salud boliviano.

1.5.2. Académica

Mediante el presente proyecto se espera dar continuidad a una serie de proyectos presentados en los últimos años en la carrera de Ingeniería Electrónica de la Universidad Mayor

de San Andrés, que aplican herramientas de inteligencia artificial para solucionar problemas o aportar en otras áreas de manera transversal, en este caso en la medicina, utilizando técnicas que se encuentran en el estado del arte como lo son el uso de redes neuronales convolucionales para clasificar imágenes médicas, el uso de estrategias de Machine Learning como lo es por ejemplo la transferencia de aprendizaje (Transfer Learning) y técnicas de análisis de datos como lo es el aumento de datos (Data Augmentation), dejando un antecedente en el uso de estas tecnologías para tareas de clasificación a partir de imágenes en general.

1.5.3. Tecnológica

El presente proyecto propone la implementación de una solución basada en inteligencia artificial para contrarrestar la ausencia de herramientas tecnológicas accesibles y efectivas que puedan integrarse al proceso de diagnóstico clínico de la neumonía. En el contexto internacional, el uso de modelos de aprendizaje profundo en el ámbito de la salud ha demostrado ser una de las estrategias más avanzadas en la detección y clasificación de patologías mediante imágenes médicas, consolidándose como el estado del arte en este campo. Este proyecto no solo adopta dichas tecnologías de vanguardia, sino que también promueve su aplicación en el sistema de salud boliviano, tomando como un objetivo del proyecto la validación de un prototipo con un profesional del sistema de salud boliviano, contribuyendo a la modernización de los métodos diagnósticos y a la difusión de estas tecnologías entre los profesionales de la salud.

1.6. Alcances y límites

1.6.1. Alcances

- Se realizará un estudio sobre el proceso de diagnóstico de neumonía basado en radiografías de tórax en Bolivia, considerando la normativa vigente y el testimonio de profesionales del sistema de salud. Este análisis permitirá contextualizar la

propuesta del sistema de visión artificial dentro del flujo de trabajo actual de los médicos y especialistas en el país.

- Se analizarán proyectos actuales que utilicen tecnologías de inteligencia artificial para el diagnóstico de neumonía, enfocándose en soluciones que empleen redes neuronales convolucionales (CNNs).
- Se investigarán diferentes arquitecturas de redes neuronales convolucionales (CNNs) que permitan realizar la tarea de clasificación de imágenes radiográficas. Se seleccionará la arquitectura más adecuada considerando su desempeño y adaptabilidad al contexto del proyecto.
- Se identificarán y seleccionarán conjuntos de datos de radiografías de tórax etiquetadas (diagnóstico positivo o negativo de neumonía). Estos datos serán procesados, incluyendo normalización y subdivisión en conjuntos de entrenamiento, validación y prueba.
- Se entrenará el modelo utilizando el conjunto de datos procesado, ajustando hiperparámetros como número de épocas y tamaño de lote. El modelo será evaluado mediante métricas estándar (precisión, exhaustividad, AUC, entre otras), iterando en función de los resultados obtenidos hasta alcanzar el mejor desempeño posible.
- Se desarrollará un prototipo de aplicación móvil para Android que integre el modelo entrenado, permitiendo a los usuarios médicos cargar imágenes radiográficas, realizar inferencias y obtener diagnósticos con su probabilidad respectiva. La aplicación será validada mediante pruebas realizadas por al menos un usuario (personal médico, especialista o radiólogo), quien comparará los

resultados con diagnósticos tradicionales y ofrecerán retroalimentación para mejorar el sistema.

- El prototipo estará diseñado para procesar imágenes de radiografías de tórax digitales preferentemente obtenidas directamente de dispositivos radiográficos. Para un mejor rendimiento del modelo se recomienda el uso de imágenes con bajo nivel de ruido, brillo y contraste adecuados, en las que la placa radiográfica ocupe la totalidad del ancho y alto de la imagen, capturada en orientación vertical.

1.6.2. Limites

Como límites del presente proyecto se consideran los siguientes aspectos:

- Acerca del conjunto de datos a ser seleccionado:
 - Los algoritmos de Aprendizaje Profundo (Deep Learning) necesitan miles o decenas de miles de muestras para llegar a generalizar (obtener resultados óptimos ante el mayor tipo de entradas posibles) y otorgar resultados más precisos. En una tarea de clasificación médica como la que es presentada en el presente proyecto ello representa un límite, debido a que una gran mayoría si no es que en todos los conjuntos de datos el número de datos ronda los cientos y en algunos casos los miles de muestras, esto sucede debido a que para el etiquetado de imágenes médicas son necesarios expertos entrenados. Esta limitante se verá reflejada en las métricas obtenidas durante la evaluación al modelo.
 - En una revisión previa, no se encontraron conjuntos de datos a nivel nacional, lo que representa una limitante. Por ello, se realizará una búsqueda de datos a nivel internacional en trabajos de investigación que hayan

recopilado una cantidad significativa de imágenes etiquetadas de radiografías de tórax.

- La complejidad física de las personas incluidas en el conjunto de datos internacional seleccionado, al no ser bolivianas, podría diferir de la complejidad común en nuestro país (entorno donde se probará el prototipo), lo que podría afectar su desempeño final.
- La neumonía como se expuso anteriormente puede ser causada por diferentes especies microbianas, el presente proyecto estará limitado a la detección de la enfermedad en función al conjunto de datos encontrado, dependiendo de este los tipos de neumonías que vayan a ser detectadas.
- No se diseñará ni desarrollará una arquitectura de red neuronal desde cero. Se empleará una arquitectura preexistente y ampliamente aceptada en el estado del arte, como ResNet-50, adaptándola a los objetivos del proyecto.
- Debido a restricciones en el hardware disponible, no se emplearán recursos computacionales de alta gama, como GPUs dedicadas de última generación. El entrenamiento del modelo, al involucrar operaciones matriciales intensivas, demanda un alto consumo de recursos computacionales, especialmente de la GPU. Por lo tanto, la selección de hiperparámetros, como el tamaño de lote, deberá ajustarse a estas limitaciones, lo que podría afectar el tiempo de entrenamiento y el rendimiento del modelo.

2. Capítulo II: Marco teórico

2.1. Fundamentos de la Detección de Neumonía con Deep Learning

2.1.1. Imágenes Médicas y Neumonía

2.1.1.1. Introducción a la neumonía

La neumonía es una enfermedad común causada por diferentes especies microbianas como bacterias, virus y hongos, esta infección provoca la inflamación de uno o ambos parénquimas pulmonares. Sin embargo, otras causas de neumonía incluyen la aspiración de alimentos y la exposición a productos químicos. Según la infección, la neumonía se produce como resultado de la inflamación causada por patógenos que hacen que los alvéolos de los pulmones se llenen de líquido o pus y, por lo tanto, provocan una disminución del intercambio de dióxido de carbono (CO₂) y oxígeno (O₂) entre la sangre y los pulmones, lo que hace que a las personas infectadas les cueste respirar [6]. La mayoría de los pacientes presentan fiebre, escalofríos, síntomas pulmonares (tos, disnea, producción de esputo, dolor torácico pleurítico) y uno o más infiltrados/opacidades en la radiografía de tórax [24].

A continuación, se detallan características propias de esta enfermedad como lo son la clasificación (un detalle de los tipos de neumonía), el diagnóstico y la evaluación. Cabe destacar que en la literatura se encuentran varias clasificaciones y varios pasos para realizar el diagnóstico y la evaluación de la neumonía variando de autor en autor, en los siguientes subtítulos se tomarán en cuenta los correspondientes al libro: Cunha, B. A. (2010). *Pneumonia Essentials 2010*. Jones & Bartlett Publishers. (2010) [24]

2.1.1.1.1. Clasificación

La neumonía se puede clasificar en términos generales en neumonía adquirida en la comunidad (NAC), neumonía adquirida en residencias de ancianos (nursing home-acquired pneumonia) (NHAP) o neumonía nosocomial (adquirida en hospital) (NN) (Véase Figura 2)

1. **Neumonía Adquirida en la Comunidad (NAC).** La NAC suele clasificarse en neumonía típica o atípica. La NAC típica se presenta sin hallazgos extrapulmonares (p. ej., miringitis, faringitis, erupción cutánea).

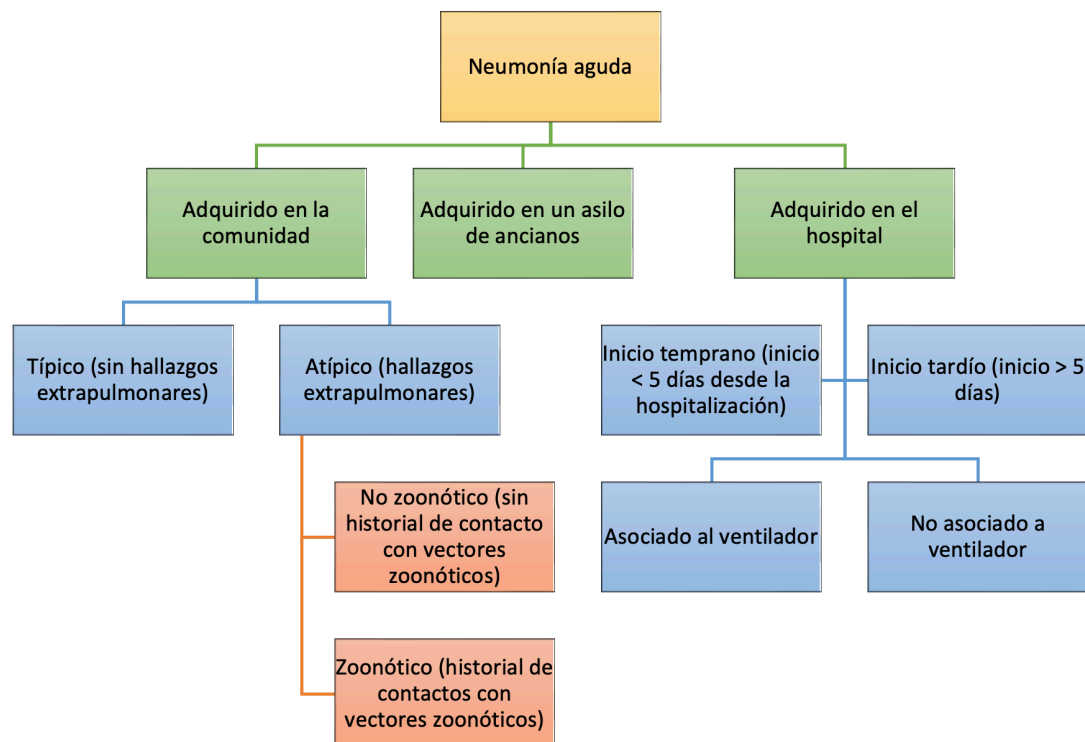
La NAC atípica se presenta con hallazgos extrapulmonares y puede clasificarse además en neumonía zoonótica o no zoonótica según la presencia de un vector zoonótico.

2. **Neumonía adquirida en residencias de ancianos (NHAP).** NHAP ocurre esporádicamente o como parte de un brote en un hogar de ancianos.

3. **Neumonía nosocomial (adquirida en el hospital) (NP).** La NP generalmente se clasifica en neumonía asociada a ventilador (NAV) o neumonía no asociada a ventilador.

Las diversas clasificaciones tienen implicaciones para el diagnóstico etiológico, la evaluación, la terapia y el pronóstico.

Figura 2. *Clasificación general de la neumonía aguda*



Nota. Adaptado del libro Cunha, B. A. (2010). *Pneumonia Essentials 2010*. Jones & Bartlett Publishers. (2010) [24]

2.1.1.1.2. Diagnóstico y evaluación

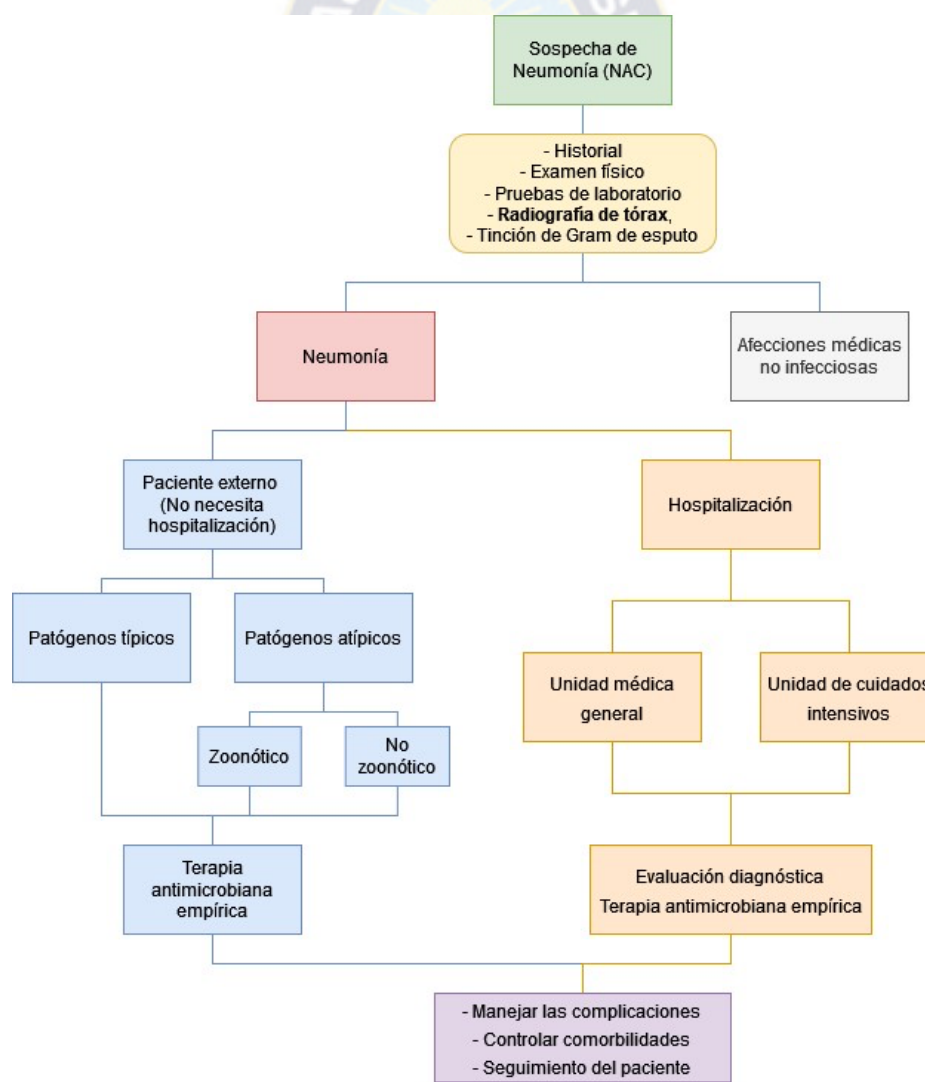
Para un diagnóstico y evaluación de un paciente con sospecha de Neumonía adquirida en la comunidad (NAC) se deben tomar en cuenta los siguientes aspectos (Véase Figura 3):

- 1. Evaluación Inicial.** En casi todos los individuos con sospecha de neumonía están indicados una anamnesis, un examen físico, pruebas de laboratorio, **una radiografía de tórax** y una tinción de Gram de esputo. Esta información se utiliza para confirmar el diagnóstico, identificar complicaciones tempranas, evaluar condiciones médicas comórbidas y llegar a un diagnóstico etiológico presuntivo. Luego se clasifica a los pacientes entre atención domiciliaria u hospitalaria y se inicia la terapia antimicrobiana empírica.
- 2. Afecciones médicas no infecciosas.** Pueden presentarse con síntomas pulmonares, fiebre y opacidades/infiltrados en la radiografía de tórax que simulan una neumonía. Los síntomas pulmonares en ausencia de opacidad/infiltrado en la radiografía de tórax generalmente se deben a una bronquitis aguda, una exacerbación aguda de la bronquitis crónica, traqueobronquitis o asma.
- 3. Gestión ambulatoria vs gestión hospitalaria.** La decisión de tratar la neumonía en un entorno ambulatorio o en un entorno hospitalario se basa en la presencia de factores de riesgo que limitan la seguridad o la eficacia del tratamiento ambulatorio, el índice de gravedad de la neumonía y el criterio clínico.
- 4. Identificación del presunto patógeno o síndrome.** Se pueden utilizar varios métodos para identificar el patógeno de la neumonía, incluido el cultivo (sangre, esputo, líquido/tejido pleural, muestra del tracto respiratorio inferior, tejido pulmonar); Respuesta de IgM o aumento de 4 veces en el título de anticuerpos contra un antígeno

microbiano; detección de antígenos (orina, sangre, líquido pleural); o amplificación de ADN/ARN microbiano (PCR) obtenido de sangre, esputo, líquido pleural, tejido o nasofaringe.

5. **Evaluación hospitalaria.** La evaluación hospitalaria tiene como objetivo identificar el patógeno de la neumonía, detectar complicaciones pulmonares y sistémicas, y evaluar el estado de las afecciones médicas comórbidas.

Figura 3. *Abordaje general de la neumonía adquirida en la comunidad en anfitriones normales*



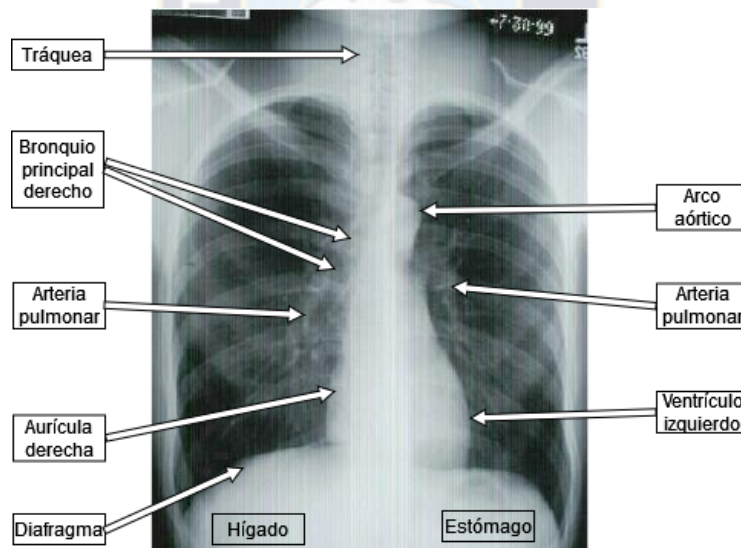
Nota. Adaptado del libro Cunha, B. A. (2010). *Pneumonia Essentials 2010*. Jones & Bartlett Publishers. (2010) [24]

2.1.1.2. Imágenes de rayos X en la detección de neumonía

Como se vio en el anterior subtítulo existen diferentes enfoques para el diagnóstico de la neumonía, algunos de estos enfoques incluyen radiografías de tórax, tomografías computarizadas, pruebas de esputo, oximetría de pulso, toracocentesis, análisis de gases en sangre, broncoscopia, cultivo de líquido pleural, hemograma completo, etc. [6] [24]

Las imágenes de rayos X de tórax son el método clínico más conocido y común para el diagnóstico de neumonía [20], para detectar enfermedades pulmonares asociadas, implicar a un patógeno (en algunos casos), y evaluar la gravedad de la enfermedad.

Figura 4. Vista posteroanterior (PA) de una radiografía de tórax con algunos elementos que son tomados en cuenta el momento de evaluar la placa radiográfica



Nota. Adaptado de X-ray Atlas: Chest X-ray <https://www.glowm.com/atlas-page/atlasid/chestXray.html> [27]

Las radiografías de tórax son una representación bidimensional (2-D) de una estructura tridimensional (3-D) y la superposición de estructuras anatómicas que se superponen con regiones de interés dificultan la detección de anomalías, incluso para radiólogos experimentados [25]. La interpretación correcta de las radiografías de tórax es compleja y subjetiva, y los diferentes lectores

se ven influenciados por la experiencia, la formación profesional y factores momentáneos como la fatiga, la distracción y la concentración [26].

2.1.1.2.1. Vistas de radiografías de torax

- **Vista anteroposterior (AP):** En una radiografía de tórax anteroposterior (AP), el haz de rayos X pasa desde la parte frontal del cuerpo (anterior) hacia la parte posterior (posterior). Esta vista se utiliza normalmente cuando los pacientes no pueden ponerse de pie o necesitan ser examinados en posición supina o semierecta, como aquellos que están gravemente enfermos o en unidades de cuidados intensivos.
- **Vista posteroanterior (PA):** En una radiografía de tórax posteroanterior (PA), el haz de rayos X pasa desde la parte posterior del cuerpo (posterior) hacia el frente (anterior). Esta vista es el método estándar y preferido para la mayoría de las radiografías de tórax, ya que proporciona la representación más precisa de la anatomía del tórax.

2.1.1.3. Terminología Radiológica para la Neumonía

Desde 1984 los miembros de la Sociedad Fleischer han publicado un glosario en inglés con los términos apropiados para la descripción de los hallazgos en la radiología del tórax. [46] Este fue denominado: “Glosario de términos para imágenes torácicas de la Sociedad de Fleischner” y fue tomado en cuenta en el set de datos 2 (Véase la sección 3.3.1.1.).

La cuarta edición del glosario de términos para imágenes torácicas de la Sociedad Fleischner es una actualización integral de las versiones anteriores de 1984, 1996 y 2008. Este glosario fue revisado para incluir nuevos términos y conceptos, eliminar los obsoletos y abordar el uso inconsistente de ciertos términos, con el objetivo de estandarizar la terminología en radiología torácica. El glosario enfatiza la importancia clínica y aclara términos que pueden ser vagos o ambiguos para mejorar la comunicación entre radiólogos y otros médicos. Admite

terminología más precisa necesaria para informes estructurados y búsquedas precisas en bases de datos.

Este glosario pretende ser una propuesta de consenso en lugar de reglas estrictas, lo que refleja la naturaleza cambiante de la terminología radiológica. Su objetivo es ser un recurso útil para los radiólogos y la comunidad de medicina respiratoria en general, contribuyendo a una mejor comunicación y comprensión en imágenes torácicas.

A continuación se expondrán algunos conceptos descritos en el mencionado glosario, mismos que también fueron usados por los profesionales que realizaron el etiquetado del conjunto de datos 2:

- **Nódulo:** Opacidad de cualquier contorno, relativamente esférica, de 3 cm o menos, rodeada por lo menos en 2/3 de su circunferencia por aire [46]
- **Atelectasia:** Reducción en la distensión de parte o de todo un pulmón. Se observa disminución en el volumen pulmonar acompañada de un incremento en la opacidad (RX) o atenuación (TC) en la parte afectada del pulmón. Con frecuencia se asocia con desplazamiento anormal de cisuras, bronquios, vasos, diafragma, corazón o mediastino. [46]
- **Masa (Patología):** Una masa es cualquier lesión circunscrita mayor de 30 mm de diámetro. El término suele referirse a una estructura de tejido blando predominantemente sólida, pero pueden estar presentes componentes cavitarios, quísticos o calcificados en diversos grados. Los descriptores de forma y margen son similares a los utilizados para los nódulos pulmonares. Además, puede haber distorsión o invasión de estructuras adyacentes. [46]
- **Colapso (Patología):** En el pulmón, sinónimo de atelectasia[46]

- **Neumonía (Patología):** Término diagnóstico de uso común que típicamente designa la infección del parénquima pulmonar. La neumonía puede ser causada por bacterias, virus, hongos, parásitos u otros microorganismos y puede ser focal o multifocal. La neumonía puede manifestarse como opacidad, potencialmente acompañada de broncogramas aéreos, opacidades en árbol en yema o derrames pleurales. [46]
- **Opacidad (Anatomía/Patología):** La opacidad se refiere a cualquier área inespecífica focal o difusa de mayor atenuación. El término es un descriptor general que no indica la naturaleza de la condición que causa la opacidad. El término se ha utilizado en ocasiones como sinónimo del ahora obsoleto y no recomendado término “infiltrado”. [46]

2.1.1.4. Sistemas de diagnóstico asistido por computadora (CAD)

Entendiendo entonces que las imágenes médicas son parte fundamental del diagnóstico de la neumonía el encontrar métodos que apoyen el análisis de las mismas es una tarea de investigación constante. Es así que los sistemas de diagnóstico asistido por computadora (CAD) forman parte de este conjunto de métodos brindando potenciales beneficios en la asistencia a personal médico en la detección de neumonía. A continuación, se describirá una breve evolución de técnicas de detección de neumonía asistidas por computadora, desde métodos de análisis de imágenes tradicional hasta el uso de Deep Learning

2.1.1.4.1. Métodos tradicionales de procesamiento de imágenes

Antes de la investigación y avances en Machine Learning y Deep Learning usualmente se tomaban en cuenta enfoques y métodos de procesamiento de imágenes para la detección de neumonía, algunos de estos fueron:

Análisis de texturas: Este tipo de métodos analiza patrones visuales en los tejidos pulmonares para identificar anomalías que sean indicativas de neumonía. Un ejemplo del mismo es el descrito en el artículo: “Textural features for image classification” de Haralick [28] publicado en 1973 donde describe varias características de textura que se calculan a partir de las matrices de coocurrencia de niveles de gris (GLCM, por sus siglas en inglés). Es así que estas características de textura de Haralick fueron estudiadas en años posteriores para la detección de anormalidades pulmonares, un ejemplo puede verse en el estudio publicado por Zayed en 2015 [29] donde se aplica este análisis de texturas a tomografías computarizadas de los pulmones.

Análisis de formas y de contornos: Estos métodos tenían como objetivo identificar características en las imágenes radiológicas como la forma de las opacidades o consolidaciones pulmonares, que son síntomas comunes de neumonía. En 2001 Van Ginneken [30] utilizó el método de segmentación: ‘Active Shape Model (ASM) Segmentation’ desarrollado por Cootes y Taylor [31] en 1995, para que combinando con métodos de extracción de características texturales y clasificación basada en vecinos más cercanos (k nearest neighbors) pueda producir una puntuación de anormalidad por cada imagen, detectando así anormalidades en radiografías frontales de tórax.

Métodos basados en la intensidad: Estos métodos analizan los valores de intensidad de los píxeles en imágenes de rayos X con el fin de identificar tejido pulmonar sano de las áreas afectadas por neumonía. Un ejemplo de ello es el trabajo presentado por R.Helen (2011) [33] que propone una modificación del método de umbralización que Otsu presentó en 1979 [32]. El método de Otsu es una técnica utilizada en procesamiento de imágenes para segmentar una imagen en dos clases: fondo y objeto. Basándose en esta técnica R.Helen propone un algoritmo 2D de Otsu basado en la optimización por enjambre de partículas (PSO) para segmentar imágenes de tomografía

computarizada (CT) de pulmones. Este método puede extraer el parénquima pulmonar de imágenes CT multi-slice, lo cual es un paso primario para detectar enfermedades pulmonares como el cáncer de pulmón, tumores y células masivas.

Estos métodos tradicionales a menudo requerían una importante experiencia en el campo de la ingeniería de características y tenían una capacidad limitada para manejar las variaciones complejas en las imágenes de rayos X.

2.1.1.4.2. Enfoques de aprendizaje automático (Machine Learning)

A partir de un conjunto de factores (como ser la disponibilidad de conjunto de datos significativamente grandes, la disponibilidad de GPU's con más potencia de cómputo, la facilidad de acceso a plataformas de 'cloud computing' (computo en la nube) entre otros), técnicas de Machine Learning se fueron popularizando en la última década de manera general, con ello también empezaron a publicarse estudios e investigaciones que utilizan modelos de aprendizaje automático para la detección de neumonía a partir de imágenes de rayos X. Estos estudios comúnmente involucraban los siguientes pasos:

Extracción de características (Feature extraction): La extracción de características significativas de las imágenes rayos X suele ser uno de los primeros pasos al realizar la detección de neumonía usando un enfoque de Machine Learning. Esta extracción se puede realizar de varias maneras, entre ellas se encuentra la extracción mediante funciones 'hechas a mano' donde investigadores diseñan funciones basadas en conocimientos previos sobre la enfermedad y patrones radiológicos asociados con la neumonía.

Entrenamiento de clasificadores (Classifier Training): Consiste en entrenar modelos de aprendizaje automático (por ejemplo, máquinas de vectores de soporte (SVM), bosques aleatorios) para distinguir en este caso entre imágenes radiológicas con neumonía y casos sanos en función de las características extraídas.

Si bien el aprendizaje automático (Machine Learning) ofrece una mayor precisión en comparación con los métodos tradicionales, la dependencia con métodos que todavía deben realizarse con supervisión humana y/o que necesitan la revisión de expertos sigue siendo una limitación. En ese contexto a partir de la segunda década de los 2010's las investigaciones relacionadas al aprendizaje profundo (Deep Learning) y las redes neuronales convolucionales toman más importancia. Previa una introducción a los conceptos sobre conjuntos de datos profundizaremos en los conceptos de aprendizaje profundo.

2.1.2. Conjuntos de Datos

El punto de partida en un proceso de Machine Learning es un conjunto de datos, en inglés Dataset, que contiene los valores de datos medidos o recopilados representados como números, texto o imágenes, un conjunto de ejemplos que contienen características importantes que describen el comportamiento del problema a resolver. Sin embargo, hay un matiz importante: si los datos proporcionados son ruidosos o tienen una relación señal-ruido baja, ni siquiera el mejor algoritmo ayudará. Por lo tanto, deberíamos intentar construir el conjunto de datos con la mayor precisión posible.[51]

- **Datos de entrenamiento (Training data):** Los modelos en un proceso de Machine Learning (ML) se construyen utilizando los datos de entrenamiento. Estos son los datos que han sido validados e incluyen el resultado deseado. La salida o los resultados generalmente se denominan etiquetas. Los datos de entrenamiento etiquetados ayudan a un modelo de ML a identificar tendencias y patrones clave esenciales para predecir el resultado posterior. [51]
- **Datos de prueba (Testing data):** Una vez entrenado el modelo, se debe evaluar su precisión. Esto se hace mediante los datos de prueba, donde el resultado generado

por ML se compara con el resultado deseado. Si ambos coinciden, entonces las pruebas han pasado. Es importante que tanto los conjuntos de datos de entrenamiento como los de prueba se parezcan a las situaciones en que los algoritmos de Machine Learning se enfrentarán más adelante. [51]

2.1.2.1. Características de los conjuntos de datos

Los conjuntos de datos ChestX-ray8 y NIH ChestX-ray14 son tomados en cuenta en la sección de ingeniería de proyecto (Véase la sección 3.3.1.1.), en el conjunto de datos 2, a continuación se detallan algunas de sus características:

- **ChestX-ray8:** Este conjunto de datos contiene 108,948 imágenes frontales de rayos X de tórax de 32,717 pacientes recopiladas de los Sistemas de comunicación y archivo de imágenes (PACS por sus siglas en inglés) del NIH Clinical Center ubicado en Bethesda, Maryland, Estados Unidos. Abarcando una amplia gama de condiciones torácicas. Las etiquetas se obtuvieron mediante técnicas de procesamiento de lenguaje natural (PLN) aplicadas a los informes radiológicos [41].
- **NIH ChestX-ray14:** Es un subconjunto de ChestX-ray8 que incluye 112,120 imágenes frontales de rayos X de tórax de 30,805 pacientes. Las etiquetas se refinaron y se centraron en 14 enfermedades torácicas comunes [43].

Ambos conjuntos de datos fueron recopilados en el Centro Clínico NIH (National Institutes of Health), ubicado en Bethesda, Maryland, Estados Unidos. El NIH es el hospital más grande del mundo dedicado exclusivamente a la investigación clínica. Como recurso nacional, el Centro Clínico desempeña un papel fundamental en la rápida traducción de observaciones científicas y descubrimientos de laboratorio en enfoques innovadores para diagnosticar, tratar y prevenir

enfermedades. Esta integración de la investigación clínica y de laboratorio tiene como objetivo cerrar la brecha entre los avances científicos y la atención al paciente, garantizando que los avances en la ciencia médica beneficien la salud pública de manera rápida y efectiva. [47]

2.1.2.2. DICOM

Los conjuntos de datos anteriormente descritos contienen imágenes en formato DICOM, que significa Digital Imaging and Communications in Medicine (Imágenes Digitales y Comunicaciones en Medicina), es un estándar integral utilizado a nivel mundial para almacenar, intercambiar y transmitir imágenes médicas y la información relacionada. El estándar DICOM facilita la interoperabilidad entre dispositivos de imágenes médicas, como escáneres y servidores, y sistemas de archivo y comunicación de imágenes (PACS por sus siglas en inglés). Esta interoperabilidad asegura que las imágenes y los datos asociados puedan compartirse sin problemas entre diferentes entornos y sistemas de atención médica, mejorando la eficiencia y efectividad de los diagnósticos y tratamientos médicos. [45]

Para el entrenamiento que se realizará será necesario convertir estas imágenes DICOM en un formato JPG, para ello es necesario entender que mientras que DICOM es el estándar para imágenes médicas, JPG es un formato de compresión de imágenes comúnmente utilizado para fotografías y gráficos en la web y en dispositivos de consumo. Es por ello que existen diferencias que deben ser entendidas:

- **Metadatos:** DICOM incluye una gran cantidad de metadatos junto con la imagen, como información del paciente, tipo de estudio, fecha de adquisición, parámetros del equipo, etc. JPG, por otro lado, contiene metadatos limitados, principalmente información básica de la imagen como tamaño y resolución.

- **Compresión:** DICOM utiliza compresión sin pérdida o con pérdida, pero prioriza la preservación de la fidelidad de la imagen para un diagnóstico preciso. JPG utiliza compresión con pérdida, lo que reduce el tamaño del archivo pero puede sacrificar algunos detalles de la imagen.
- **Escala de grises:** DICOM a menudo utiliza una escala de grises de 12 o 16 bits, lo que permite una mayor gama de valores de intensidad y detalles sutiles en las imágenes médicas. JPG generalmente usa una escala de grises de 8 bits, adecuada para imágenes generales, pero con menos detalles para uso médico.

2.1.2.3. Diseño de Estudio: Cohorte Retrospectiva

El conjunto de datos 1 (Véase la sección 3.3.1.1.), se basa en un diseño de estudio de cohorte retrospectiva. Un estudio de cohorte retrospectivo es un tipo de estudio observacional comúnmente utilizado en epidemiología e investigación médica. El diseño de este estudio implica identificar y seguir a un grupo de individuos (cohorte) que comparten una característica o experiencia común dentro de un período de tiempo definido. La característica clave de un estudio retrospectivo es que el resultado o los eventos de interés ya han ocurrido en el momento en que comienza el estudio. Los investigadores retroceden en el tiempo para examinar las exposiciones a factores de riesgo o de protección sospechosos en relación con un resultado establecido al inicio del estudio. [49]

2.1.3. Aprendizaje Profundo (Deep Learning)

El aprendizaje profundo es un subcampo del aprendizaje automático (Machine learning) que se enfoca en el desarrollo de modelos computacionales compuestos por múltiples capas de procesamiento interconectadas. Estas capas aprenden representaciones jerárquicas de los

datos, donde cada nivel de abstracción se construye sobre el anterior, permitiendo descubrir patrones y estructuras intrincadas en grandes conjuntos de datos. [57]

A diferencia de los enfoques tradicionales de aprendizaje automático, el aprendizaje profundo no requiere una ingeniería de características manual, sino que aprende automáticamente las representaciones más relevantes para la tarea en cuestión. Esto se logra mediante el uso del algoritmo de retropropagación (backpropagation algorithm), que ajusta los parámetros internos de la red neuronal en función del error cometido en la predicción, mejorando iterativamente su rendimiento.

El aprendizaje profundo ha revolucionado diversos campos, logrando avances significativos en áreas como:

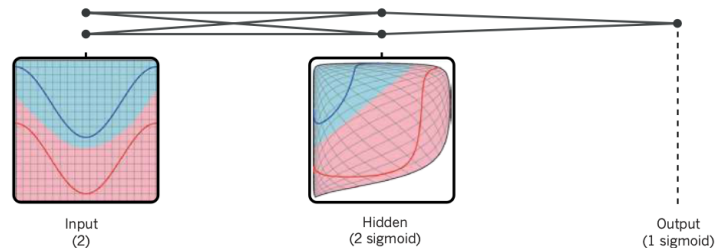
- **Reconocimiento de voz:** Mejora la precisión en la transcripción y comprensión del lenguaje hablado.
- **Reconocimiento visual de objetos y detección de objetos:** Permite identificar y localizar objetos en imágenes y videos con alta precisión.
- **Descubrimiento de fármacos y genómica:** Facilita el análisis de grandes volúmenes de datos biológicos para identificar posibles dianas terapéuticas y desarrollar nuevos medicamentos.
- **Procesamiento de lenguaje natural:** Mejora la traducción automática, el resumen de textos y la generación de lenguaje natural.

Entre los tipos de redes neuronales más utilizados en el aprendizaje profundo se encuentran:

- **Redes convolucionales profundas (CNN):** Especializadas en el procesamiento de imágenes, video, voz y audio, aprovechando la estructura espacial de los datos.

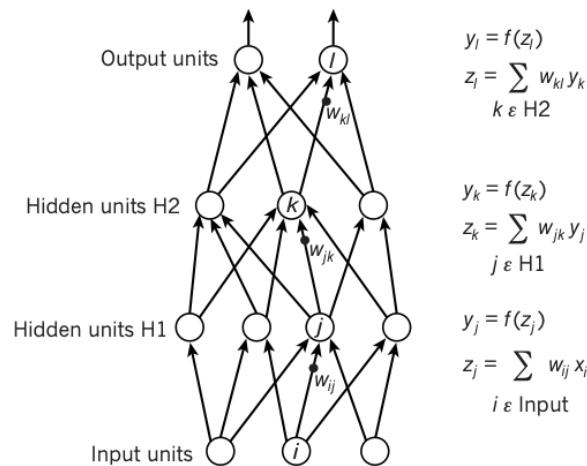
- **Redes recurrentes profundas (RNN):** Diseñadas para procesar secuencias de datos, como texto y habla, capturando dependencias temporales y relaciones a largo plazo.

Figura 5. *Distorsión del espacio de entrada*



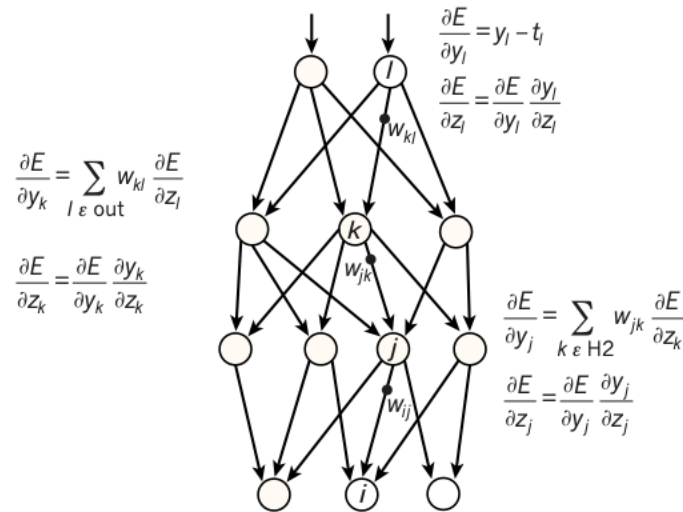
Nota. La figura muestra cómo una red neuronal con capas ocultas (hidden layers) puede transformar el espacio de entrada para hacer que las clases de datos (representadas por líneas rojas y azules) sean linealmente separables. En otras palabras, la red aprende a "enderezar" los datos para facilitar su clasificación. Basado en LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. nature, 521(7553), 436-444. [57]

Figura 6. *Propagación hacia adelante (forward pass)*



Nota. La figura detalla las ecuaciones utilizadas para calcular la salida de una red neuronal a partir de una entrada dada. Cada unidad en una capa recibe una suma ponderada de las salidas de las unidades de la capa anterior, y luego aplica una función no lineal (como la sigmoide o ReLU) para obtener su propia salida. Basado en LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. nature, 521(7553), 436-444. [57]

Figura 7. Retropropagación (backward pass)



Nota. Muestra cómo se calculan los gradientes de error en cada capa de la red neuronal. Estos gradientes indican cómo deben ajustarse los pesos de las conexiones entre las neuronas para reducir el error de predicción. La retropropagación comienza en la capa de salida, donde se compara la salida de la red con los valores objetivo, y luego se propaga hacia atrás a través de las capas ocultas. Basado en LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. nature, 521(7553), 436-444. [57]

2.1.3.1. Redes Neuronales Convolucionales (ConvNets)

Las ConvNets son un tipo especializado de arquitectura de aprendizaje profundo diseñadas para procesar datos en forma de múltiples matrices, como imágenes, espectrogramas de audio y videos [57]. A diferencia de las redes neuronales tradicionales, que tratan cada entrada de forma independiente, las ConvNets aprovechan la estructura espacial de los datos para aprender patrones locales y jerárquicos.

2.1.3.1.1. Principios clave de las ConvNets

Las redes neuronales convolucionales aprovechan cuatro ideas clave para explotar las propiedades de las señales naturales:

- **Conexiones locales:** Cada unidad en una capa convolucional se conecta a un pequeño parche de unidades en la capa anterior, capturando patrones y características locales, como bordes o texturas.

- **Pesos compartidos:** Todas las unidades en un mapa de características comparten el mismo banco de filtros, detectando el mismo patrón en diferentes partes de la matriz. Esto reduce el número de parámetros y mejora la generalización.
- **Pooling:** Las capas de pooling combinan características semánticamente similares en una sola, proporcionando un grado de invariancia a la traducción y reduciendo la dimensionalidad de la representación.
- **Múltiples capas:** Las ConvNets generalmente constan de varias etapas de capas convolucionales y de pooling, seguidas de capas completamente conectadas para clasificación o regresión.

2.1.3.1.2. Componentes de una ConvNet

La arquitectura de una ConvNet típica consta de lo siguiente:

- **Capa de entrada:** Recibe los datos de entrada sin procesar (por ejemplo, una imagen).
- **Capas convolucionales:** Aplican filtros a la imagen para extraer características. Cada filtro produce un mapa de características que resalta un patrón específico en la imagen.
- **Funciones de activación:** Introducen no linealidad en el modelo. Una función de activación común es la ReLU (Rectified Linear Unit), que activa una neurona solo si su entrada es positiva. En este proyecto, se utiliza una variante de la ReLU llamada Leaky ReLU, que permite un pequeño gradiente incluso cuando la entrada es negativa.
- **Capas de pooling:** Reducen la dimensionalidad de los mapas de características. Los tipos más comunes de pooling son el max pooling y el average pooling.

- **Capas lineales:** También conocidas como capas densas o completamente conectadas, combinan las características extraídas por las capas convolucionales para realizar la clasificación. Cada neurona en una capa lineal está conectada a todas las neuronas de la capa anterior.
- **Capa de clasificación:** La última capa lineal, llamada capa de clasificación, produce las salidas para las diferentes clases. En este proyecto, la capa de clasificación tiene dos neuronas de salida, una para la clase "neumonía" y otra para la clase "normal"

2.1.3.1.3. *Aplicaciones de las ConvNets*

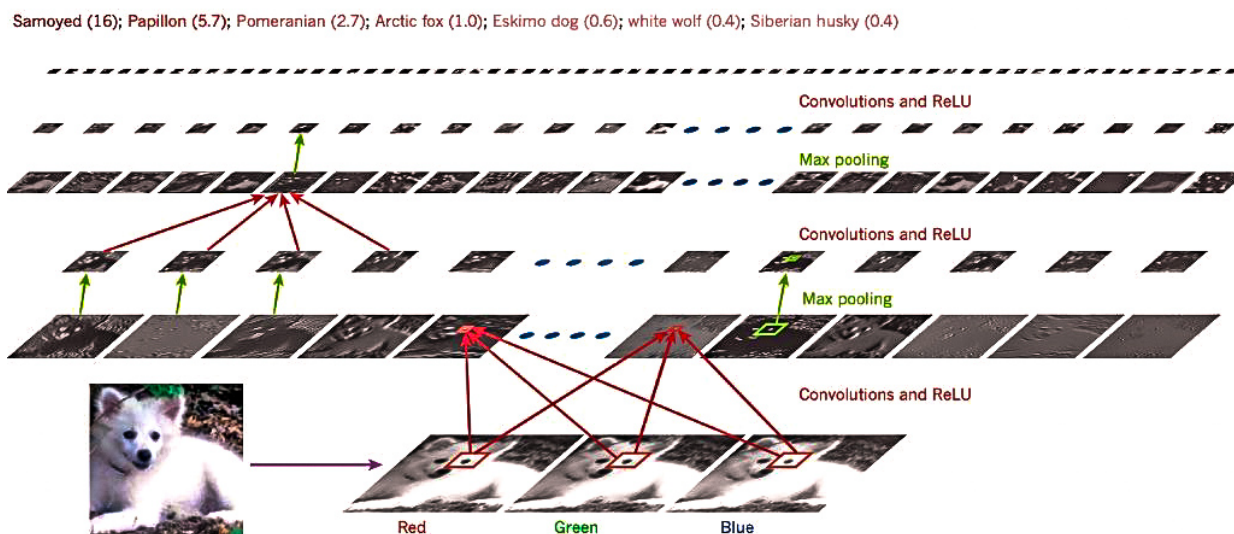
Las ConvNets se han aplicado con éxito a una amplia gama de tareas, que incluyen:

- **Clasificación de imágenes:** Identificar la categoría de una imagen.
- **Detección de objetos:** Localizar objetos dentro de una imagen.
- **Segmentación de imágenes:** Etiquetar cada píxel en una imagen con la clase del objeto al que pertenece.
- **Reconocimiento facial:** Identificar a las personas a partir de sus rasgos faciales.

Además de sus aplicaciones prácticas, las ConvNets también están inspiradas biológicamente, inspirándose en la corteza visual del cerebro. El concepto de capas convolucionales y capas de pooling recuerda a las células simples y complejas en la neurociencia visual.

El éxito de las ConvNets ha llevado a su adopción generalizada en varios campos, incluida la visión por computadora, el procesamiento del lenguaje natural y la robótica. Continúan siendo un área importante de investigación y desarrollo en el aprendizaje profundo, con nuevas arquitecturas y aplicaciones que surgen regularmente.

Figura 8. *Por dentro de una red neuronal convolucional*



Nota. En la figura se puede ver todo el proceso que sigue una red convolucional: se inicia con la imagen, en este caso de un perro Samoyedo (en la esquina inferior izquierda), a su derecha, los canales rojo, verde y azul (RGB) se muestran por separado. Estos son los valores de píxeles sin procesar que la red recibe como entrada. A partir de allí, las capas convolucionales aplican filtros que detectan características simples, como bordes o texturas. La función de activación ReLU introduce no linealidad, ayudando a la red a aprender relaciones complejas. Posteriormente, las capas de max pooling reducen las dimensiones espaciales de los mapas de características para hacer la red menos sensible a pequeñas variaciones. A medida que se avanza en las capas, las características se vuelven más abstractas, llegando a representar partes del perro, como ojos o pelaje. Finalmente, la red produce una puntuación para cada clase posible, siendo la clase con la puntuación más alta la predicción final del contenido de la imagen. Basado en LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, 521(7553), 436-444. [57]

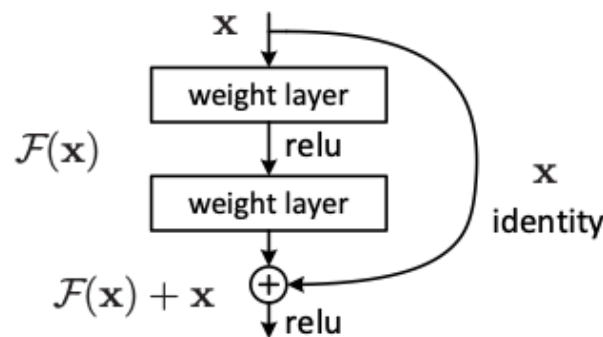
2.1.3.2. ResNet (Red Residual) y ResNet 50

Las redes neuronales profundas han revolucionado el reconocimiento de imágenes. Sin embargo, simplemente apilar más capas no siempre conduce a un mejor rendimiento. De hecho, las redes más profundas pueden sufrir el problema de degradación, donde la precisión se estanca y luego disminuye a medida que aumenta la profundidad. Esto no se debe al sobreajuste (overfitting), sino a las dificultades en el entrenamiento de estos modelos complejos.

ResNet (Red Residual) es una arquitectura innovadora diseñada para abordar este problema de degradación. Introduce el concepto de aprendizaje residual, donde las capas aprenden funciones residuales (la diferencia entre la entrada y la salida) en lugar de funciones no referenciadas [58]. Esto se logra a través de conexiones de salto (skip o shortcut connections), que son rutas directas

que omiten una o más capas. Estas conexiones de salto permiten que la información fluya más fácilmente a través de la red, mitigando el problema del gradiente que desaparece y permitiendo el entrenamiento de redes mucho más profundas.

Figura 9. Conexión de salto (Aprendizaje residual)



Nota. Adaptado de: He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. [58]

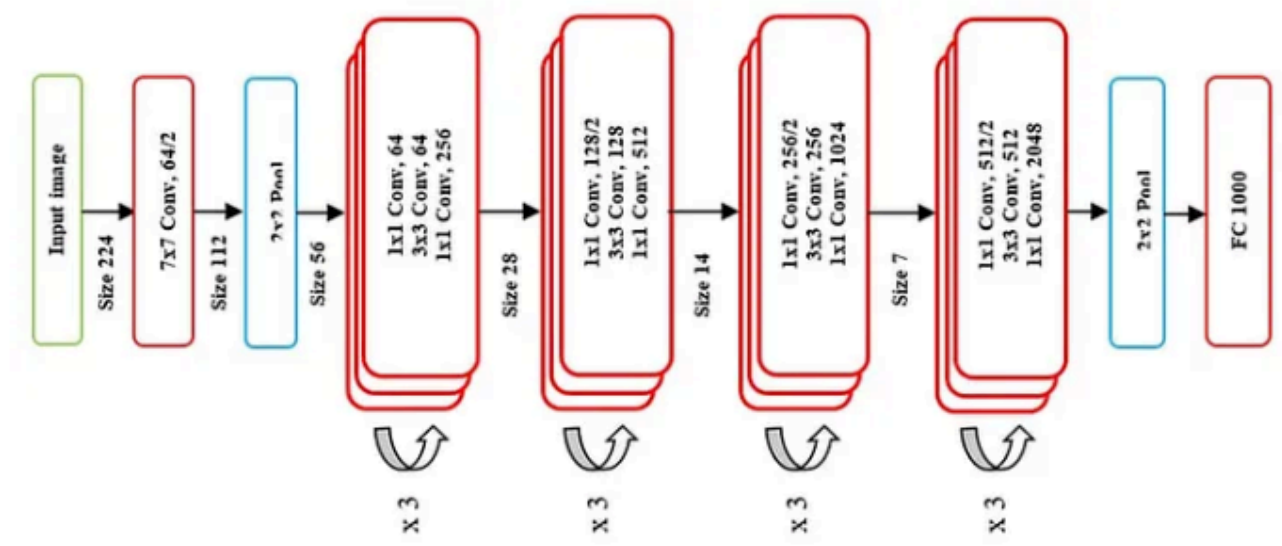
2.1.3.2.1. ResNet-50

ResNet-50 es una configuración específica de ResNet con 50 capas. Es una opción popular debido a su equilibrio entre profundidad y eficiencia computacional.

La arquitectura consta de:

- **Preprocesamiento de Entrada:** Capas iniciales para estandarizar la imagen de entrada.
- **Bloques Cfg:** Estos son los componentes básicos de la red, cada uno con múltiples capas convolucionales. ResNet-50 utiliza diferentes números de bloques Cfg en diferentes etapas para controlar la complejidad del modelo.
- **Capa Completamente Conectada:** La capa final que agrega información y produce las predicciones de salida.

Figura 10. *Arquitectura ResNet-50*



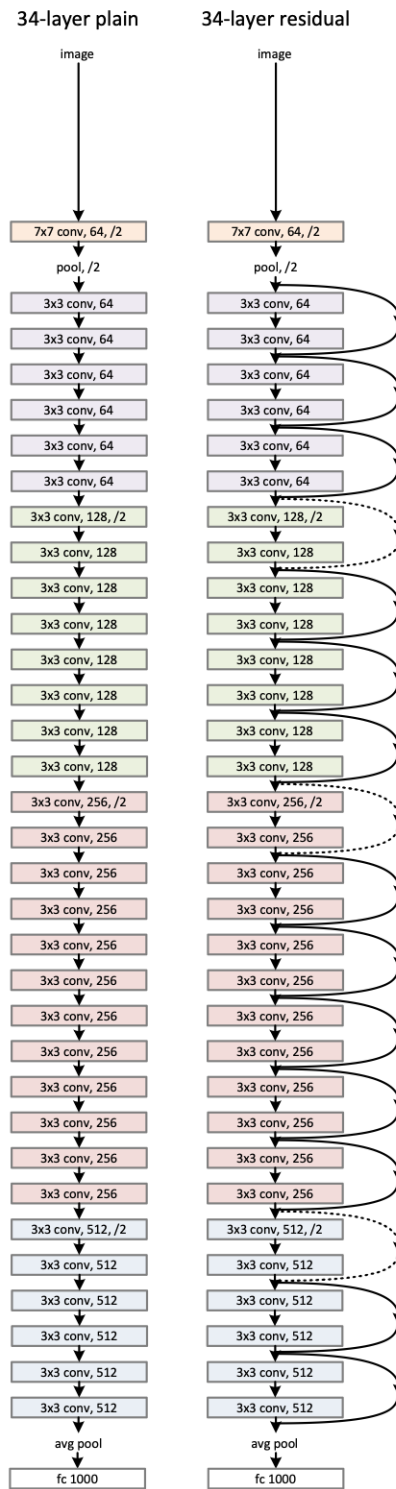
Nota. Extraído de Mukherjee, S. (2022). The annotated resnet-50: Explaining how ResNet-50 works and why it is so popular, Towards Data Science [59]

2.1.3.2.1. ResNet-50 en la Detección de Neumonía en Imágenes de Rayos X

Según los conceptos indicados anteriormente, y aplicándolos al presente proyecto podemos indicar que ResNet-50 es adecuado para la detección de neumonía en imágenes de rayos X:

- **Representaciones Profundas:** La profundidad de ResNet-50 le permite aprender características intrincadas en las imágenes de rayos X, cruciales para distinguir patrones sutiles asociados con la neumonía.
- **Reutilización de Características:** Las conexiones de salto permiten la reutilización de características de capas anteriores, lo cual puede ser especialmente útil al tratar con las texturas complejas y opacidades sutiles que a menudo se observan en casos de neumonía.
- **Reducción del Sobreajuste:** El marco de aprendizaje residual y las conexiones de salto ayudan a prevenir el sobreajuste, asegurando que el modelo se generalice bien a nuevas imágenes de rayos X no vistas.

Figura 11. Ejemplos de arquitecturas de redes neuronales para el set de datos ImageNet.

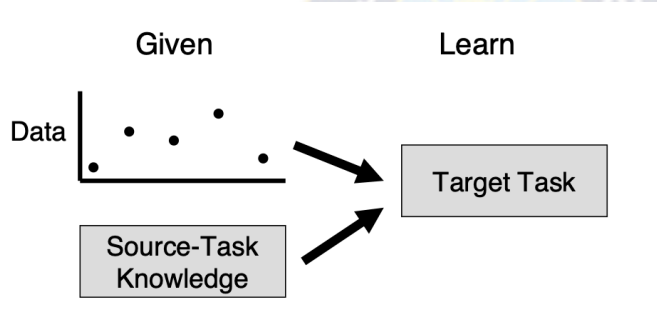


Nota. A la izquierda un ejemplo de una red simple, a la derecha una red residual. Figura adaptada de: He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. [58]

2.1.3.3. Transferencia de aprendizaje

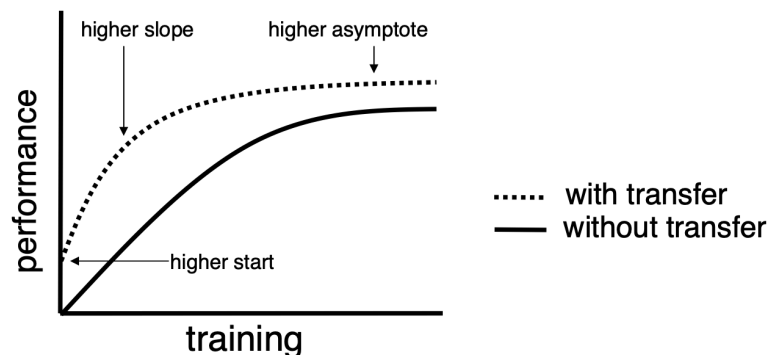
La transferencia de aprendizaje es un paradigma del aprendizaje automático que busca mejorar el proceso de aprendizaje de una nueva tarea utilizando el conocimiento adquirido de una tarea relacionada y previamente aprendida. Este enfoque refleja la forma intuitiva en que los humanos aplican el conocimiento de experiencias pasadas para enfrentar nuevas situaciones. Cuanto más se alinee la nueva tarea con experiencias previas, más suave será la curva de aprendizaje.[56]

Figura 12. *La transferencia de aprendizaje y el aprendizaje automático*



Nota. La transferencia de aprendizaje es un enfoque de aprendizaje automático que utiliza una fuente adicional de información además de los datos de entrenamiento estándar: el conocimiento de una o más tareas relacionadas. Extraído de Torrey, L., & Shavlik, J. (2010). Transfer learning. In Handbook of research on machine learning applications and trends: algorithms, methods, and techniques (pp. 242-264). IGI global. [56]

Figura 13. *Tres maneras en las cuales el método de transferencia mejora el aprendizaje*



Nota. Extraído de Torrey, L., & Shavlik, J. (2010). Transfer learning. In Handbook of research on machine learning applications and trends: algorithms, methods, and techniques (pp. 242-264). IGI global. [56]

Este paradigma tiene algunas características u objetivos citados a continuación que son los que se pretenden obtener tras su aplicación:

- **Mejora del Rendimiento Inicial:** La transferencia de aprendizaje busca elevar el rendimiento inicial en una tarea objetivo aprovechando el conocimiento transferido, incluso antes de que ocurra cualquier nuevo aprendizaje. Esto contrasta con el rendimiento inicial de un agente que carece de cualquier conocimiento relevante previo.
- **Tiempo de Aprendizaje Acelerado:** Al transferir conocimiento, el tiempo requerido para aprender completamente la tarea objetivo puede reducirse significativamente en comparación con el aprendizaje desde cero.
- **Rendimiento Final Mejorado:** La transferencia de aprendizaje tiene como objetivo lograr un nivel de rendimiento final más alto en la tarea objetivo en comparación con lo que sería posible sin transferir conocimiento.

Así también su aplicación tiene algunos desafíos:

- **Transferencia Negativa:** Un desafío clave es garantizar una transferencia positiva, donde el rendimiento mejora, evitando al mismo tiempo la transferencia negativa, donde el rendimiento se deteriora debido a la aplicación inapropiada del conocimiento.
- **Mapeo de Tareas:** Es crucial identificar y establecer correspondencias entre las características de las tareas de origen y objetivo. Aunque a menudo se hace manualmente, algunos métodos buscan automatizar este proceso.

2.1.3.4. ImageNet

ImageNet es una extensa base de datos de imágenes diseñada para la investigación en reconocimiento visual de objetos. Contiene más de 14 millones de imágenes que han sido

manualmente anotadas para indicar qué objetos aparecen en ellas. Las imágenes están organizadas según la jerarquía de WordNet, un diccionario léxico en inglés que agrupa palabras en conjuntos de sinónimos (synsets). Cada imagen está etiquetada con los objetos presentes en ella, lo que permite entrenar modelos de aprendizaje automático para reconocer esos objetos. El tamaño masivo de ImageNet lo convierte en un recurso invaluable para entrenar modelos de aprendizaje profundo que requieren grandes cantidades de datos etiquetados.[61]

2.1.3.4.1. Modelo ResNet-50 pre-entrenado con ImageNet

Un modelo ResNet-50 pre-entrenado con ImageNet es una instancia de la arquitectura ResNet-50 que ya ha sido entrenada en el conjunto de datos ImageNet. Esto significa que el modelo ha aprendido a extraer características generales de las imágenes y a clasificarlas en 1000 categorías diferentes de objetos.

Utilizar un modelo pre-entrenado ofrece varias ventajas:

- **Ahorro de tiempo y recursos:** No es necesario entrenar el modelo desde cero, lo que reduce el tiempo de entrenamiento y los recursos computacionales necesarios.
- **Mejor rendimiento:** El modelo pre-entrenado ya ha aprendido características relevantes de las imágenes, lo que puede mejorar el rendimiento en nuevas tareas de visión artificial.
- **Generalización:** El modelo pre-entrenado ha sido expuesto a una gran variedad de imágenes, lo que puede mejorar su capacidad de generalización a nuevos datos.

2.1.3.5. Data augmentation

El aumento de datos se refiere a un conjunto de técnicas utilizadas para mejorar la diversidad y el volumen de datos de entrenamiento para modelos de aprendizaje automático, en particular modelos de aprendizaje profundo, mediante la creación de datos sintéticos. Estos datos

sintéticos se generan a partir del conjunto de datos existente a través de varias **transformaciones** que introducen pequeños cambios preservando las características esenciales de los datos. El objetivo principal del aumento de datos es mejorar el rendimiento y la generalización de los modelos evitando el sobreajuste y permitiéndoles aprender características más sólidas e invariantes.[50]

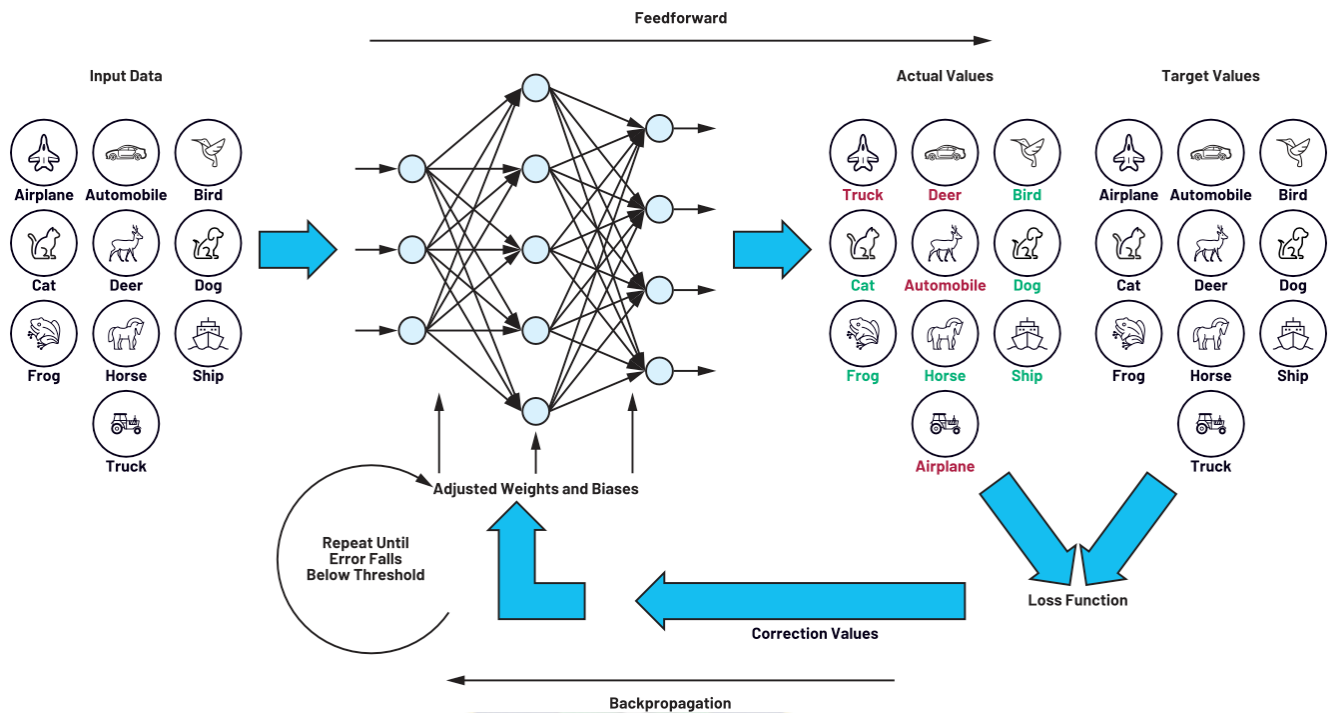
2.1.3.6. Entrenamiento de Redes Neuronales Convolucionales

El entrenamiento de una red neuronal convolucional (ConvNet) es el proceso mediante el cual la red aprende a reconocer patrones y características en los datos de entrada. Este proceso implica ajustar los pesos de las conexiones entre las neuronas para minimizar el error en la predicción. [71] Algunos aspectos importantes a tomar en cuenta son:

- **Datos de entrenamiento:** Para entrenar una ConvNet, se requiere un conjunto de datos de entrenamiento que consiste en imágenes etiquetadas. Las etiquetas indican la clase a la que pertenece cada imagen. Por ejemplo, en un conjunto de datos de imágenes de rayos X, las etiquetas podrían ser "neumonía" o "normal".
- **Proceso de entrenamiento:** El entrenamiento de una ConvNet se basa en el principio de backpropagation (propagación hacia atrás). El proceso se puede resumir en los siguientes pasos:
 1. **Feedforward:** Se presenta una imagen al modelo y se propaga a través de las capas de la red, generando una predicción.
 2. **Cálculo del error:** Se compara la predicción del modelo con la etiqueta real de la imagen, calculando el error o pérdida.
 3. **Backpropagation:** El error se propaga hacia atrás a través de la red, ajustando los pesos de las conexiones entre las neuronas para minimizar el error.

4. **Repetición:** Los pasos 1 a 3 se repiten para cada imagen del conjunto de entrenamiento, y el proceso se itera varias veces (épocas) hasta que el modelo converge a un rendimiento aceptable.

Figura 14. Ciclo de entrenamiento describiendo *feedforward* y *backpropagation*



Nota. Figura obtenida de Dressen, O. (n.d.). *Training Convolutional Neural Networks: What Is Machine Learning? Part 2.* <https://www.analog.com/en/resources/analog-dialogue/articles/training-convolutional-neural-networks-what-is-machine-learning-part-2.html> [71]

- **Función de pérdida:** La función de pérdida mide el error entre la predicción del modelo y la etiqueta real. Existen diferentes tipos de funciones de pérdida, y la elección depende de la tarea específica. En este proyecto, se utiliza la función de pérdida CrossEntropyLoss, que es común en problemas de clasificación.
- **Algoritmo de optimización:** El algoritmo de optimización se encarga de ajustar los pesos de la red para minimizar la función de pérdida. Existen diferentes

algoritmos de optimización, como el descenso de gradiente estocástico (SGD), Adam, RMSprop, etc. En este proyecto, se utiliza el optimizador Adam.

- **Sobreajuste y subajuste:** Durante el entrenamiento, es importante evitar el sobreajuste (overfitting) y el subajuste (underfitting).
 - **Sobreajuste:** Ocurre cuando el modelo se ajusta demasiado a los datos de entrenamiento y no es capaz de generalizar a nuevos datos. Se puede identificar cuando la pérdida en el conjunto de validación aumenta mientras que la pérdida en el entrenamiento sigue disminuyendo.
 - **Subajuste:** Ocurre cuando el modelo no es lo suficientemente complejo para aprender los patrones en los datos. Se puede identificar cuando la pérdida tanto en el entrenamiento como en la validación es alta.

Consideraciones adicionales (otros hiperparámetros):

- **Tamaño del lote (batch size):** El tamaño del lote determina cuántas imágenes se procesan en cada iteración del entrenamiento.
- **Tasa de aprendizaje (learning rate):** La tasa de aprendizaje controla la magnitud de los ajustes en los pesos de la red.
- **Número de épocas:** El número de épocas determina cuántas veces se procesa el conjunto de datos de entrenamiento completo.

El entrenamiento de una ConvNet es un proceso iterativo que requiere la selección cuidadosa de hiperparámetros y la monitorización del rendimiento del modelo para evitar el sobreajuste y el subajuste.

2.1.3.7. Evaluación del modelo

La evaluación del modelo es una etapa crítica en el desarrollo de un sistema de detección de neumonía. Permite medir la eficacia del modelo y asegurar que cumpla con los requisitos de rendimiento deseados. Este proceso implica el uso de métricas de evaluación apropiadas y un análisis cuidadoso de los resultados para comprender las fortalezas y debilidades del modelo.

2.1.3.7.1. Métricas de evaluación

Las métricas de evaluación proporcionan una forma cuantitativa de medir el rendimiento del modelo. La elección de las métricas depende de la naturaleza del problema y de los objetivos específicos del sistema. Algunas métricas comunes utilizadas en la detección de neumonía incluyen:

- **Exactitud (Accuracy):** Esta métrica mide la proporción de predicciones correctas sobre el total de predicciones. Se calcula como:

$$\text{Exactitud} = (TP + TN) / (TP + FP + TN + FN) \quad (1)$$

Donde:

- TP representa los verdaderos positivos
 - TN representa los verdaderos negativos
 - FP representa los falsos positivos
 - FN representa los falsos negativos.
- **Precisión (Precision):** La precisión mide la proporción de predicciones positivas correctas sobre el total de predicciones positivas. Se calcula como:

$$\text{Precision} = TP / (TP + FP) \quad (2)$$

- **Sensibilidad (Recall) o Exhaustividad:** La sensibilidad, también conocida como recall, mide la proporción de predicciones positivas correctas sobre el total de casos positivos reales. Se calcula como:

$$Recall = TP / (TP + FN) \quad (3)$$

- **Puntuación F1 (F1-score):** La puntuación F1 es una medida armónica de precisión y sensibilidad, que proporciona un equilibrio entre ambas métricas. Se calcula como:

$$F1 - score = 2 / ((1 / Precision) + (1 / Recall)) \quad (4)$$

- **Área bajo la curva ROC (AUC):** El área bajo la curva ROC es una medida de la capacidad del modelo para distinguir entre las clases. La curva ROC representa la relación entre la tasa de verdaderos positivos y la tasa de falsos positivos para diferentes umbrales de clasificación.

2.1.3.7.2. *Matriz de Confusión*

La matriz de confusión es una tabla que permite visualizar el desempeño del modelo en la clasificación de cada clase. Muestra el número de verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos. La matriz de confusión proporciona una visión detallada de los tipos de errores que comete el modelo, lo que puede ser útil para identificar áreas de mejora.

2.1.3.7.3. *Curvas de aprendizaje (learning curves)*

Las curvas de aprendizaje, con raíces en la psicología educativa y conductual/cognitiva, son representaciones gráficas que ilustran el proceso de aprendizaje de un modelo de Machine Learning a lo largo del tiempo, medido en épocas. Al igual que en el aprendizaje humano, donde el esfuerzo y el tiempo necesarios para aprender aumentan con la complejidad de la tarea, en Machine Learning, las curvas de aprendizaje reflejan cómo la capacidad de predicción del modelo (precisión) evoluciona con la experiencia (cantidad de datos procesados en cada época).

Cada época representa un ciclo completo de entrenamiento con el conjunto de datos, simulando el proceso iterativo de aprendizaje humano. A medida que el modelo procesa más datos a través de las épocas, adquiere experiencia y ajusta sus parámetros para realizar predicciones más

precisas. Las curvas de aprendizaje proporcionan una visualización de este progreso, permitiendo monitorear el aprendizaje y detectar posibles problemas como el sobreajuste.

2.2. Herramientas de Desarrollo e Implementación

2.2.1. Python

Python se ha convertido en un lenguaje común de muchas aplicaciones de ciencia de datos. Combina el poder de los lenguajes de programación de propósito general con la facilidad de uso de lenguajes de programación de dominios específicos como MATLAB o R.

Python tiene bibliotecas para carga de datos, visualización, estadísticas, procesamiento de lenguaje natural, procesamiento de imágenes y más. Esta amplia caja de herramientas proporciona a los científicos de datos una amplia gama de funciones generales y especiales. Una de las principales ventajas de usar Python es la capacidad de interactuar directamente con el código, usando una terminal u otras herramientas como Jupyter Notebook. El aprendizaje automático (Machine learning) y el análisis de datos son procesos fundamentalmente iterativos, en los que los datos impulsan el análisis. Es fundamental que estos procesos cuenten con herramientas que permitan una rápida iteración y una fácil interacción.

Como lenguaje de programación de propósito general, Python también permite la creación de interfaces gráficas de usuario (GUI) y servicios web complejos, y la integración en sistemas existentes.[52]

2.2.2. PyTorch

PyTorch es una biblioteca para Python que facilita la construcción de proyectos de aprendizaje profundo. Enfatiza la flexibilidad y permite que los modelos de aprendizaje profundo se expresen en un Python idiomático, lo que lo hace accesible y fácil de usar. Esta facilidad de uso encontró adopción temprana en la comunidad de investigación y, desde su primer lanzamiento,

PyTorch se ha convertido en una de las herramientas de aprendizaje profundo más prominentes en una amplia gama de aplicaciones.[53]

Así como Python proporciona una excelente introducción a la programación, PyTorch ofrece una excelente introducción al aprendizaje profundo. Su sintaxis clara, API simplificada y facilidad de depuración lo convierten en una opción ideal para aquellos nuevos en el campo. Aunque PyTorch es amigable para principiantes, también está totalmente calificado para trabajos profesionales, del mundo real y de alto perfil.

En su núcleo, PyTorch es un framework de aprendizaje profundo que relaciona entradas a salidas utilizando funciones matemáticas complejas. Para facilitar esto, PyTorch proporciona una estructura de datos central llamada tensor, que es un array multidimensional. Construido sobre esta base, PyTorch incluye características para realizar operaciones matemáticas aceleradas en hardware dedicado, lo que lo hace conveniente para diseñar y entrenar arquitecturas de redes neuronales tanto en máquinas individuales como en recursos de computación paralela.

2.2.2.1. Herramientas de PyTorch

- **Torchvision:** Es una biblioteca es parte del proyecto PyTorch. El paquete Torchvision consta de conjuntos de datos populares, arquitecturas de modelos y transformaciones de imágenes comunes para la visión artificial. [54]
- **Torchvision transforms:** Torchvision admite transformaciones de visión artificial comunes en los módulos Torchvision.transforms y Torchvision.transforms.v2. Las transformaciones se pueden utilizar para transformar o aumentar datos para el entrenamiento o la inferencia de diferentes tareas (clasificación de imágenes, detección, segmentación, clasificación de videos), incluyendo el cambio de tamaño, la normalización. [55]

En el presente proyecto se utilizarán varias clases del módulo Transforms de Torchvision:

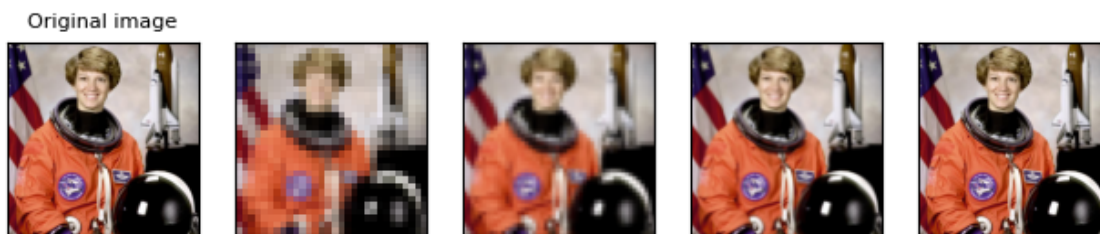
- **torchvision.transforms.Compose:** Permite encadenar múltiples transformaciones de imágenes en una sola operación.

Ejemplo:

```
transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor()
])
```

- **torchvision.transforms.Resize:** Ajusta las dimensiones de la imagen a un tamaño uniforme, que a menudo se requiere para la entrada del modelo. (Véase la Figura 15)
- **torchvision.transforms.RandomHorizontalFlip:** Voltea la imagen horizontalmente al azar con una probabilidad dada. (Véase la Figura 16)
- **torchvision.transforms.RandomVerticalFlip:** Voltea la imagen verticalmente al azar con una probabilidad dada. (Véase la Figura 17)
- **torchvision.transforms.RandomAffine:** Aplica transformaciones afines aleatorias a la imagen, como rotación, traslación, escalado y cizalladura. (Véase la Figura 18)
- **torchvision.transforms.ToTensor:** Convierte una imagen PIL o un array numpy.ndarray en un tensor. Este método es esencial para convertir imágenes en un formato que puedan procesar los modelos de PyTorch.

Figura 15. Resultado de la clase la aplicación de la clase *torchvision.transforms.Resize*



Nota. Extraído de Illustration of transforms — Torchvision 0.11.0 documentation. (s. f.).
https://pytorch.org/vision/0.11/auto_examples/plot_transforms.html#sphx-glr-auto-examples-plot-transforms-py
[55]

Figura 16. Resultado de la aplicación de la clase *torchvision.transforms.RandomHorizontalFlip*

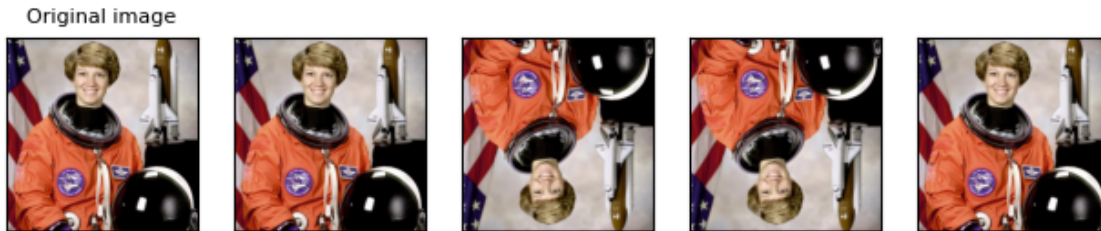


Nota. Extraído de Illustration of transforms — Torchvision 0.11.0 documentation. (s. f.).
https://pytorch.org/vision/0.11/auto_examples/plot_transforms.html#sphx-glr-auto-examples-plot-transforms-py
[55]

- **torchvision.transforms.Normalize:** Normaliza una imagen tensor con una media y desviación estándar. Escala los valores de los píxeles a un rango estándar, mejorando la convergencia del modelo durante el entrenamiento.
- **torch.utils.data:** Proporciona utilidades para cargar datos, como las clases Dataset y DataLoader. Dataset es una clase abstracta que representa un conjunto de datos, mientras que DataLoader envuelve un iterable alrededor del Dataset para permitir el fácil acceso a los datos durante el entrenamiento y la evaluación.
- **torchvision.models:** Proporciona un conjunto de modelos pre-entrenados que se pueden utilizar para diversas tareas de visión por computadora. Los modelos en este

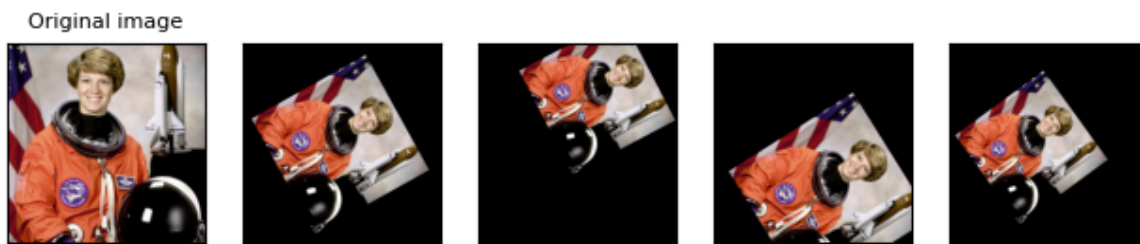
módulo han sido entrenados en grandes conjuntos de datos de referencia como ImageNet y son capaces de lograr alta precisión en varias tareas. [60]

Figura 17. Resultado de la aplicación de la clase *torchvision.transforms.RandomVerticalFlip*



Nota. Extraído de Illustration of transforms — Torchvision 0.11.0 documentation. (s. f.).
https://pytorch.org/vision/0.11/auto_examples/plot_transforms.html#sphx-glr-auto-examples-plot-transforms-py
[55]

Figura 18. Resultado de la aplicación de la clase *torchvision.transforms.RandomAffine*



Nota. Extraído de Illustration of transforms — Torchvision 0.11.0 documentation. (s. f.).
https://pytorch.org/vision/0.11/auto_examples/plot_transforms.html#sphx-glr-auto-examples-plot-transforms-py
[55]

- **torchvision.models.resnet50:** Devuelve una instancia del modelo ResNet-50, que es una red neuronal convolucional con 50 capas. Es posible entregarle el parametro booleano pretrained en True, este parámetro asegura que el modelo se inicialice con pesos pre-entrenados en el conjunto de datos ImageNet. [60]

2.2.2.2. Otras Librerías de Python

- **Numpy:** Es uno de los paquetes fundamentales para la informática científica en Python. Contiene funcionalidad para matrices multidimensionales, funciones

matemáticas de alto nivel como operaciones de álgebra lineal, la transformada de Fourier y generadores de números pseudoaleatorios.[52]

- **Matplotlib:** Es la principal biblioteca de trazado científico en Python. Proporciona funciones para realizar visualizaciones con calidad de publicación, como gráficos de líneas, histogramas, diagramas de dispersión, etc.[52]
- **Pandas:** Es una biblioteca de Python para la manipulación y el análisis de datos. Está construido alrededor de una estructura de datos llamada DataFrame que se modela a partir del R DataFrame. En pocas palabras, un DataFrame de pandas es una tabla, similar a una hoja de cálculo de Excel. Pandas proporciona una gran variedad de métodos para modificar y operar en esta tabla; en particular, permite consultas similares a SQL y uniones de tablas. A diferencia de NumPy, que requiere que todas las entradas de una matriz sean del mismo tipo, pandas permite que cada columna tenga un tipo independiente (por ejemplo, números enteros, fechas, números de punto flotante y cadenas). Otra herramienta valiosa proporcionada por pandas es su capacidad para ingerir desde una gran variedad de formatos de archivos y bases de datos, como archivos SQL, Excel y archivos de valores separados por comas (CSV). [52]

2.2.3. Desarrollo de Aplicaciones Móviles con Android

2.2.3.1. Introducción a Android

Android es uno de los sistemas operativos móviles más utilizados en la actualidad. Desarrollado por Google, se basa en el núcleo Linux y está diseñado principalmente para teléfonos inteligentes y tabletas. La naturaleza de código abierto de Android ha impulsado su rápido

crecimiento, lo que lo convierte en uno de los favoritos tanto para los consumidores como para los desarrolladores [68]. A continuación, algunas de sus características principales:

- **Código abierto:** el diseño de código abierto de Android permite a los desarrolladores modificar y mejorar fácilmente sus características para satisfacer las demandas cambiantes de la tecnología móvil. Esta flexibilidad fomenta la innovación y un ecosistema de aplicaciones diverso.
- **Amplio ecosistema de aplicaciones:** con más de 1500 millones de aplicaciones y juegos descargados de Google Play cada mes [68], los usuarios de Android tienen acceso a una enorme selección de software.
- **Potente marco de desarrollo:** Android ofrece un sólido kit de desarrollo de software (SDK) que permite a los desarrolladores crear aplicaciones para una amplia gama de dispositivos. El SDK incluye herramientas como un depurador, bibliotecas, un emulador, documentación, código de muestra y tutoriales.
- **Conectividad:** Android admite varias opciones de conectividad, como GSM, Bluetooth, Edge, 3G, 4G, Wi-Fi y GPS.

2.2.3.2. Desarrollo con Android

El SDK de Android proporciona las herramientas y los recursos necesarios para que los desarrolladores creen aplicaciones utilizando el lenguaje de programación Java. Esto incluye:

- **Depurador:** ayuda a identificar y corregir errores en el código.
- **Bibliotecas:** módulos de código preconstruidos que proporcionan funcionalidades comunes.
- **Emulador:** permite a los desarrolladores probar sus aplicaciones en un dispositivo virtual.

- Documentación, código de muestra y tutoriales: ofrecen orientación y soporte para los desarrolladores.
- La naturaleza abierta de Android, combinada con sus potentes funciones y su completo SDK, lo ha posicionado como una fuerza líder en el mundo móvil.

2.2.3.3. Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) predilecto para la creación de aplicaciones Android. Basado en IntelliJ IDEA (un IDE para Java), ofrece un conjunto de herramientas robustas que facilitan el desarrollo eficiente de apps. Emplea Gradle, un sistema de compilación flexible que se apoya en la Máquina Virtual de Java (JVM) y permite la ejecución de scripts Java dentro del IDE para un mayor control sobre el proceso de compilación [69]. Además, Android Studio utiliza Instant Push para implementar cambios en el código fuente de la aplicación en tiempo real, sin necesidad de reiniciarla, lo que acelera el ciclo de desarrollo al permitir visualizar los cambios de forma inmediata.

2.2.3.4. Lenguaje de programación Java

El lenguaje de programación Java es un lenguaje de propósito general, concurrente, basado en clases y orientado a objetos, diseñado para ser sencillo y accesible para los programadores. Aunque tiene similitudes con C y C++, Java está organizado de manera distinta, omitiendo aspectos de esos lenguajes e incorporando ideas de otros [35].

Java es un lenguaje fuertemente tipado, lo que permite identificar errores en tiempo de compilación y distingue estos de los que ocurren en tiempo de ejecución. El proceso de compilación genera código byte independiente de la máquina, y en tiempo de ejecución se gestionan la carga de clases, la generación de código y la optimización dinámica.

Java es de alto nivel, ya que abstrae detalles de la máquina y gestiona automáticamente la memoria mediante un recolector de basura, lo que evita problemas de seguridad relacionados con la desasignación explícita. No incluye características inseguras, como accesos a arreglos sin verificación, para garantizar un comportamiento predecible del programa.

2.2.3.5. Bibliotecas de Android

- **AndroidX:** AndroidX es una importante colección de bibliotecas que proporciona versiones mejoradas y actualizadas de los componentes, así como nuevas funcionalidades para el desarrollo de aplicaciones Android. Su estructura de paquetes facilita la gestión de dependencias y la compatibilidad entre diferentes versiones de Android. Esencialmente, AndroidX se ha convertido en el estándar para el desarrollo moderno de aplicaciones Android, ofreciendo una base sólida para construir apps robustas y mantenibles.
- **PyTorch Mobile:** PyTorch Mobile es un framework que permite ejecutar modelos de aprendizaje automático (machine learning) directamente en dispositivos Android. Permite integrar funcionalidades de inteligencia artificial, como reconocimiento de imágenes o procesamiento de lenguaje natural, en aplicaciones móviles.
- **Android CameraX:** CameraX es una biblioteca de Jetpack que simplifica el desarrollo de aplicaciones que utilizan la cámara del dispositivo. Abstrae las complejidades de la gestión de la cámara, ofreciendo una API consistente y fácil de usar. Proporciona componentes predefinidos para casos de uso comunes, como la vista previa, la captura de imágenes y la grabación de video.

- **AppCompat:** AppCompat es una biblioteca que proporciona compatibilidad con características más nuevas de Android en versiones anteriores del sistema operativo. Permite utilizar componentes de la interfaz de usuario (UI) modernos, como Material Design, en dispositivos con versiones antiguas de Android.
- **ConstraintLayout:** ConstraintLayout es un componente de diseño que permite crear interfaces de usuario complejas y responsivas de forma declarativa. Define las posiciones y relaciones entre los elementos de la UI mediante restricciones, lo que facilita la creación de layouts flexibles que se adaptan a diferentes tamaños de pantalla y orientaciones.
- **Material Design:** Material Design es un lenguaje de diseño desarrollado por Google que define un conjunto de principios y componentes para crear interfaces de usuario atractivas y fáciles de usar. Se basa en conceptos de diseño visual, interacción y movimiento para ofrecer una experiencia de usuario consistente e intuitiva en diferentes plataformas

2.3. Metodología de Desarrollo: Mobile-D

Los métodos ágiles han demostrado ser adecuados para el desarrollo de aplicaciones móviles. Sin embargo, las características específicas de los dispositivos móviles imponen ciertas limitaciones a la aplicación de los métodos ágiles existentes. Para abordar esto, Pekka Abrahamsson et al. (2004)[63] propusieron una nueva metodología ágil de desarrollo de aplicaciones móviles denominada Mobile-D, que se basa en prácticas ágiles y extrae elementos de métodos ágiles establecidos, como Extreme Programming (prácticas de desarrollo) y Crystal methodologies (method scalability), y el Rational Unified Process (cobertura del ciclo de vida).

La metodología Mobile-D está diseñada para empresas de desarrollo de software de tamaño pequeño y mediano. Está optimizada para un equipo de menos de diez desarrolladores que trabajan en un entorno de oficina compartido, con el objetivo de entregar aplicaciones móviles completamente funcionales en un plazo corto.

2.3.1. Fases de Mobile-D

Un proyecto de desarrollo que sigue el enfoque Mobile-D se divide en cinco iteraciones [62] o fases. Estas fases son:

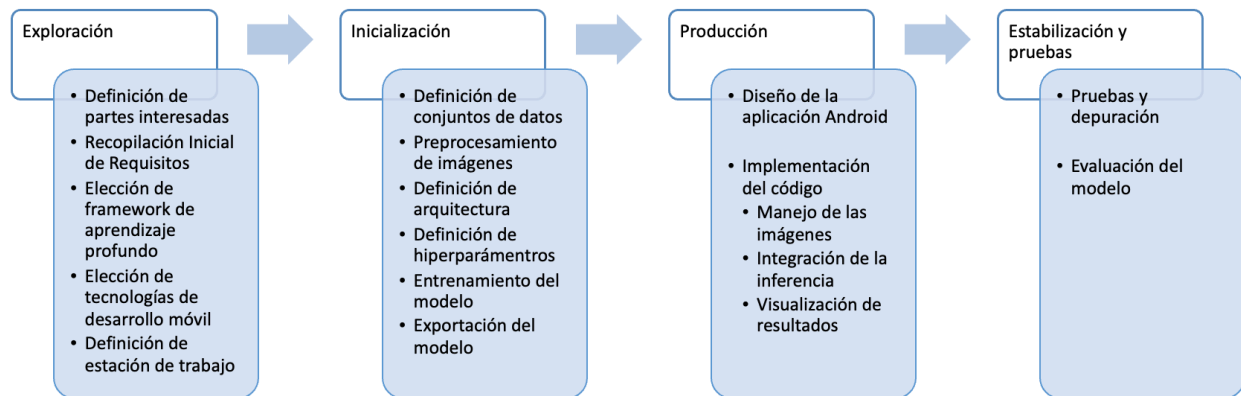
- **Exploración:** esta fase inicial implica la iniciación del proyecto y la recopilación de requisitos. Incluye la identificación de las partes interesadas, la definición del alcance del proyecto y la configuración del plan inicial del proyecto.
- **Inicialización:** la fase de inicialización se centra en la configuración del entorno de desarrollo, la creación del plan inicial del proyecto y la preparación de los recursos necesarios para el proceso de desarrollo.
- **Producción:** esta es la fase principal en la que se desarrollan y prueban la aplicación. Se divide en varias iteraciones, cada una con sus propios días de planificación, trabajo y lanzamiento.
- **Estabilización:** en la fase de estabilización, se estabiliza la aplicación y se corrigen los defectos restantes. Esta fase garantiza la calidad de la aplicación y la prepara para la fase de prueba final.
- **Prueba y reparación del sistema:** la fase final en la que la aplicación se somete a pruebas exhaustivas y se realizan las correcciones necesarias. Esta fase garantiza que la aplicación cumpla con todos los requisitos y esté lista para su lanzamiento.

3. Capítulo III: Ingeniería de proyecto

3.1. Introducción

A partir de los objetivos planteados en el capítulo 1, se desarrolló un sistema de visión artificial que utiliza técnicas de aprendizaje profundo para detectar neumonía a partir de imágenes radiológicas. Este sistema se implementó en una aplicación para dispositivos móviles Android. El prototipo final permite al usuario subir una imagen desde su galería, tomar una fotografía o utilizar una de las imágenes de prueba predefinidas, para luego realizar una inferencia con un modelo pre-entrenado, obteniendo un diagnóstico preliminar (“Tiene neumonía” o “No tiene neumonía”) y un valor numérico de probabilidad. Para el desarrollo de este proyecto se utilizó la metodología Mobile-D [62][63], que divide el proceso en las siguientes fases: Exploración, Inicialización, Producción, Estabilización y Prueba. En la Figura 19 se puede observar un resumen y secuencia de las fases anteriormente mencionadas aplicadas al presente proyecto.

Figura 19. *Diagrama resumen de la ingeniería de proyecto*



Nota. Elaboración propia

3.2. Exploración

Esta fase inicial consiste en la planificación y el establecimiento de las características del proyecto, siguiendo las tres etapas clave de Mobile-D: establecimiento de partes interesadas (stakeholders), definición del alcance y establecimiento del proyecto.

3.2.1. Establecimiento de partes interesadas

En este proyecto, se identificaron algunos grupos de partes interesadas:

- **Profesionales de la salud (médicos y/o radiólogos):** Son los usuarios directos de la aplicación y forman parte de la elaboración de los conjuntos de datos que serán utilizados para el entrenamiento del modelo de inteligencia artificial por lo que su participación en este proyecto es esencial.
- **Pacientes:** Llegan a ser usuarios indirectos del sistema obteniendo un impacto en su diagnóstico y en la atención médica.

3.2.2. Recopilación Inicial de Requisitos

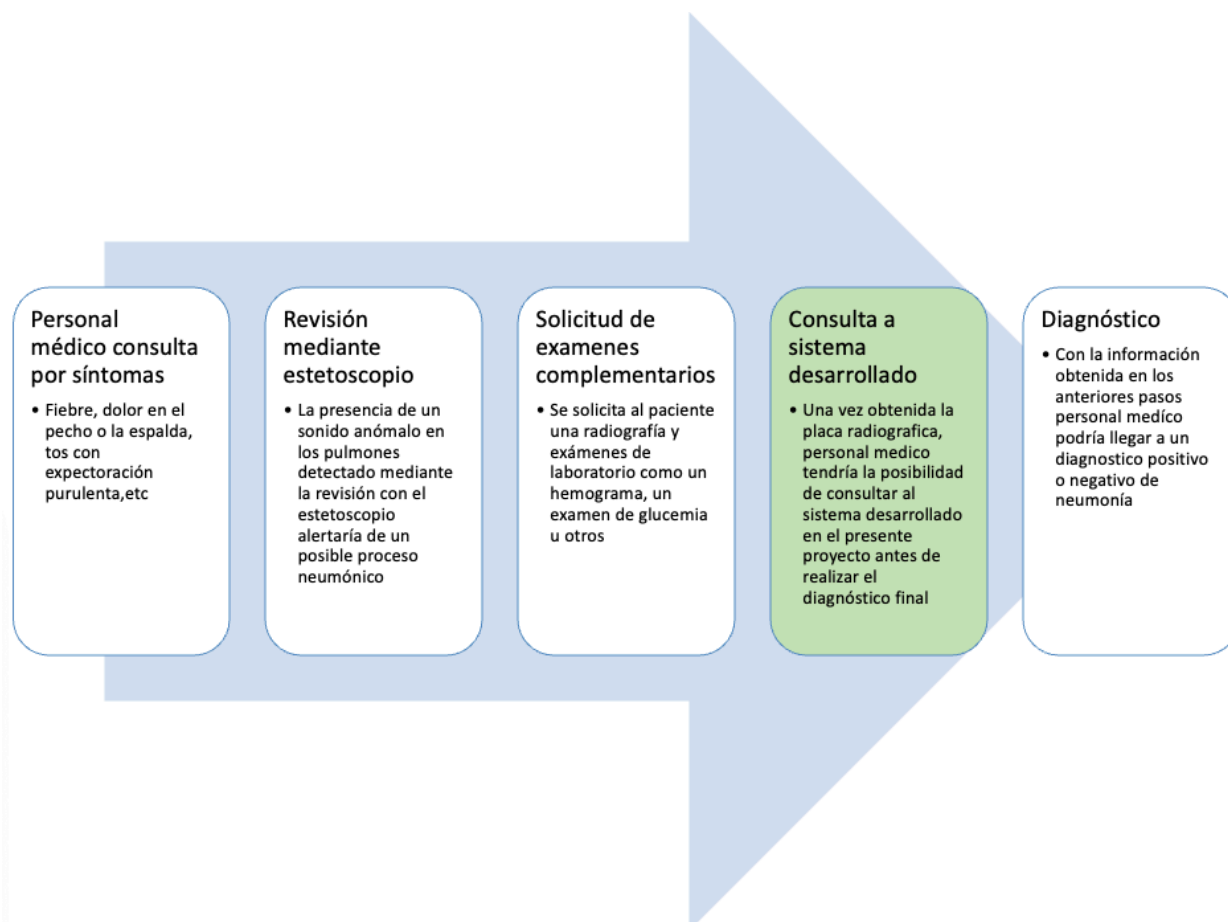
La principal tarea de esta etapa es la recopilación inicial de requisitos con el objetivo de obtener una definición clara del alcance, propósito y funcionalidad del sistema propuesto. Para ello, se identificaron aspectos clave descritos a continuación.

3.2.2.1. Análisis del Proceso Actual de Diagnóstico de Neumonía en Bolivia

En Bolivia, el diagnóstico de neumonía sigue un proceso clínico basado en la evaluación médica y en exámenes complementarios. Según la entrevista realizada al Dr. Edgar Calderón, neumólogo del Instituto Nacional de Tórax (Establecimiento de salud de tercer nivel situado en la ciudad de La Paz), el procedimiento diagnóstico se compone de las siguientes etapas (Véase Sección 1.3 y Figura 1):

- Consulta Médica Inicial
- Exploración Clínica
- Solicitud de Exámenes Complementarios
- Diagnóstico Final

Figura 20. Diagrama del proceso de diagnóstico de neumonía con la solución desarrollada



Nota. Elaboración propia basado en entrevista a Dr. Edgar Calderón

A pesar de ser un procedimiento estándar, el diagnóstico actual presenta varias limitaciones, particularmente en áreas con escasez de recursos médicos:

- **Tiempo de diagnóstico:** En hospitales con alta demanda o en zonas rurales, el acceso a especialistas puede ser limitado, generando retrasos en la evaluación de las radiografías.
- **Subjetividad en la interpretación de imágenes:** La variabilidad en la experiencia de los médicos y radiólogos puede ocasionar diferencias en la interpretación de los hallazgos radiográficos, afectando la precisión del diagnóstico.

- **Accesibilidad limitada:** En entornos rurales, la falta de especialistas y equipos de diagnóstico reduce las oportunidades de detección temprana, lo que puede derivar en complicaciones graves o un tratamiento tardío.

Para abordar estas problemáticas, se propone un sistema basado en inteligencia artificial que se integrará al proceso actual de diagnóstico (Véase la Figura 20).

3.2.2.1.1. *Análisis del sistema operativo Android*

En el marco del presente proyecto resulta fundamental seleccionar una plataforma que garantice accesibilidad y eficacia, especialmente en entornos con recursos limitados. La elección del sistema operativo Android se sustenta en los siguientes aspectos:

- **Accesibilidad y Penetración en el Mercado:** En Bolivia, Android es el sistema operativo predominante en dispositivos móviles, con una cuota de mercado que oscila entre el 90% y el 94%, [73] [74]. Esto implica que, de los aproximadamente 6.3 millones de usuarios de teléfonos inteligentes en el país [75], entre 4.7 y 4.9 millones utilizan dispositivos con Android.

Esta amplia adopción, combinada con la mayor disponibilidad y asequibilidad de los dispositivos Android en comparación con otras plataformas, permite que la aplicación tenga un mayor alcance en zonas rurales y comunidades con menores recursos, facilitando su implementación como herramienta de apoyo para la detección de neumonía.

- **Compatibilidad con Herramientas de Aprendizaje Profundo:** Las versiones modernas de Android (a partir de la versión 9 del sistema operativo) integran mejoras significativas en el rendimiento, la gestión de recursos y el soporte para frameworks de inteligencia artificial, como Pytorch Mobile. Estas optimizaciones

son cruciales para el procesamiento de imágenes radiológicas y garantizan una ejecución fluida del sistema de visión artificial, incluso en dispositivos de gama media o baja.

3.2.2.2. Requerimientos del Sistema

Los siguientes requerimientos describen las funcionalidades esenciales y las características técnicas que debe cumplir el presente sistema. Estos requisitos están organizados en cuatro categorías: funcionales, no funcionales, técnicos y requerimientos para el entrenamiento del modelo.

3.2.2.2.1. *Requerimientos Funcionales*

Los requerimientos funcionales especifican las capacidades esenciales que debe proporcionar el sistema en cada una de sus fases de operación, desde la adquisición de imágenes hasta la visualización de los resultados.

Tabla 1. *Requerimientos funcionales del prototipo (aplicación móvil)*

ID	Requerimiento	Descripción
RF1	Captura y carga de imágenes radiológicas	Permitir la captura de imágenes con la cámara del dispositivo móvil y la carga de imágenes de rayos X de tórax desde la galería del dispositivo en formatos JPG y PNG.
RF2	Preprocesamiento de la imagen	Redimensionar automáticamente las imágenes, normalizar y convertir al formato de entrada del modelo.
RF3	Clasificación de neumonía offline	Clasificar las imágenes con las etiquetas: “Tiene neumonía” / “No tiene neumonía”, utilizando un modelo preentrenado incorporado, sin necesidad de conexión a Internet.

RF4	Visualización del puntaje de confianza	Mostrar confianza en la predicción (por ejemplo 95,4 % de probabilidad de neumonía).
RF5	Modo de prueba con imágenes precargadas	Permitir la evaluación y prueba del sistema utilizando imágenes predefinidas dentro de la aplicación.

Nota. Elaboración propia

3.2.2.2.2. *Requerimientos No Funcionales*

Los requerimientos no funcionales establecen las restricciones, estándares de calidad y condiciones de uso que debe cumplir el sistema para garantizar su rendimiento óptimo y la experiencia del usuario.

Tabla 2. *Requerimientos no funcionales*

ID	Requisito	Descripción
RNF1	Precisión del modelo	La sensibilidad (Relación de predicciones positivas correctas sobre el total de casos positivos reales), debe ser mayor a 90% en la etapa de evaluación del modelo con un conjunto de datos de prueba.
RNF2	Operación sin conexión	El sistema no debe requerir conexión a Internet después de la instalación de la aplicación, ya que el modelo y la lógica estarán completamente integrados en el dispositivo.
RNF3	Compatibilidad del dispositivo	Compatible con versiones del sistema operativo Android desde 2018 en adelante (API 26+, Android 9).

Nota. Elaboración propia

3.2.2.2.3. *Requerimientos Técnicos*

Los requerimientos técnicos establecen las especificaciones mínimas necesarias para el correcto funcionamiento del sistema en términos de almacenamiento, hardware y optimización de recursos.

Tabla 3. *Requerimientos técnicos*

ID	Requisito	Descripción
RT1	Límite de tamaño del modelo	El archivo del modelo de IA debe ser ≤ 100 MB, para garantizar que pueda ser almacenado y ejecutado en dispositivos con espacio de almacenamiento limitado.
RT2	Eficiencia de los recursos de hardware	La aplicación debe estar optimizada para dispositivos con ≥ 4 GB de RAM y sin GPU dedicada, asegurando una ejecución eficiente sin consumo excesivo de memoria.
RT3	Formato de salida del modelo	El modelo de IA debe estar exportado en un formato compatible con dispositivos móviles, como TensorFlow Lite (.tflite) o PyTorch Mobile (.ptl), asegurando una inferencia eficiente.

Nota. Elaboración propia

3.2.2.2.4. *Requerimientos para el entrenamiento del modelo*

Los siguientes requerimientos establecen los criterios para la selección del conjunto de datos, el proceso de entrenamiento y evaluación del modelo de inteligencia artificial.

Tabla 4. *Requerimientos del conjunto de datos y entrenamiento del modelo*

ID	Requisito	Descripción
RCD1	Validación médica	Todas las imágenes deben estar etiquetadas por radiólogos certificados, garantizando la fiabilidad del conjunto de datos.

RCD2	Equilibrio de clases	Se debe mantener una proporción balanceada de casos de neumonía y casos normales para evitar sesgos en el modelo.
RCD3	Preprocesamiento de datos	Aplicar métodos de preprocesamiento como normalización o escalado para mejorar la calidad del entrenamiento.
RCD4	Volumen de datos	Se requiere un mínimo de 5,000 imágenes para garantizar una buena generalización del modelo.
RCD5	Aumento de datos	Implementar transformaciones como rotaciones aleatorias, traslaciones, escalado entre otras para mejorar la robustez del modelo.
RCD6	Hardware de entrenamiento	El modelo debe ser entrenado utilizando una GPU con soporte CUDA para optimizar los tiempos de entrenamiento. En caso de no disponibilidad, debe poder ejecutarse en CPU con ajuste de hiperparámetros.
RCD7	Métricas de evaluación	Evaluar el modelo utilizando AUC-ROC, precisión, recall y F1-score para medir su desempeño.

Nota. Elaboración propia

3.2.3. Establecimiento del Proyecto

El objetivo de esta etapa es definir y asignar los recursos necesarios para el desarrollo del presente proyecto. También se busca asegurar que el proceso de implementación se lleve a cabo sin retrasos derivados de la falta de herramientas, infraestructura o información técnica adecuada.

3.2.3.1. Elección de las Tecnologías

La selección de tecnologías para este proyecto se realizó tomando en cuenta los requerimientos definidos previamente, priorizando factores como:

- Compatibilidad con dispositivos móviles (RNF3).
- Eficiencia en el preprocesamiento y entrenamiento del modelo (RCD3, RCD5).
- Facilidad de integración con Android y PyTorch Mobile (RT2, RT3).
- Optimización del rendimiento y uso eficiente de recursos (RNF2, RT1, RT2).

3.2.3.1.1. Framework de Aprendizaje Profundo

La elección de PyTorch como framework de aprendizaje profundo se basó en un análisis comparativo con otras opciones, como TensorFlow. Se determinó que PyTorch era la mejor opción para este proyecto debido a las siguientes ventajas:

- **Versatilidad en el preprocesamiento de datos:** PyTorch ofrece una amplia gama de funciones de transformación de imágenes (`torchvision.transforms`) [55], que permiten aplicar técnicas como redimensionamiento, normalización, inversión horizontal/vertical y deformaciones geométricas. Esto es clave para cumplir con los requerimientos de preprocesamiento y aumentación de datos (RCD3, RCD5).
- **Documentación completa y accesible:** PyTorch ofrece una documentación bien organizada y ejemplos de implementación, lo que facilita la replicación del proceso y la optimización del modelo para su despliegue en dispositivos móviles (RT3) [65].
- **Facilidad de uso e integración:** PyTorch se caracteriza por su facilidad de uso y su integración fluida con otras herramientas y bibliotecas. Esto simplificó el desarrollo del modelo y su posterior integración en la aplicación Android.

3.2.3.1.2. Tecnologías de Desarrollo de la Aplicación

- **Entorno de Desarrollo:** Se utilizó Android Studio como entorno de desarrollo integrado (IDE) para el desarrollo de la aplicación Android. Android Studio ofrece un conjunto completo de herramientas para el diseño,

la codificación, la depuración y la creación de aplicaciones Android. Su uso permite cumplir con los requerimientos funcionales RF1-RF5, ya que facilita la implementación de funcionalidades como la captura y carga de imágenes, preprocesamiento, inferencia con IA y visualización de resultados.

- **Lenguaje de Programación:** Se utilizó Java como lenguaje de programación para el desarrollo de la aplicación. Java es un lenguaje ampliamente utilizado en el desarrollo de aplicaciones Android, con una gran comunidad de desarrolladores y una amplia gama de bibliotecas disponibles. Entre estas se encuentra PyTorch Mobile Lite, una biblioteca que permite la integración del modelo de IA en la aplicación y que es compatible con los formatos de salida del modelo definidos en el requerimiento RT3.
- **Bibliotecas utilizadas en el desarrollo de la aplicación:**
 - **CameraX:** Para la integración de la cámara y captura de imágenes de rayos X, asegurando la facilidad de uso en la interfaz de usuario.
 - **PyTorch Mobile Lite:** Permite la ejecución del modelo en dispositivos móviles sin conexión a Internet, cumpliendo con RNF2 (Operación sin conexión) y RT3 (Formato de salida del modelo).
 - **Otras bibliotecas de Android:** Para el manejo de la interfaz de usuario y otras funcionalidades.

3.2.3.1.3. Estación de Trabajo

Para el preprocesamiento de las imágenes de los conjuntos de datos, el entrenamiento del modelo, el desarrollo de la aplicación móvil y las pruebas de la aplicación se seleccionó una estación de trabajo con las siguientes características:

Tabla 5. Especificaciones estación de trabajo

Componente	Especificación	Justificación
Sistema Operativo	Windows 11 Pro 23H2	Compatibilidad con bibliotecas de IA y herramientas de desarrollo.
Procesador	Intel(R) Core(TM) i5-10600K CPU @ 4.10GHz	Suficiente para el preprocesamiento y manipulación de grandes volúmenes de imágenes.
Memoria RAM	16 GB DDR4 (2 x 8 GBytes Dual Channel 3200 MHz)	Permite la carga eficiente del conjunto de datos y la ejecución del modelo sin interrupciones.
Tarjeta Gráfica	NVIDIA GeForce RTX 2060 6GBytes VRAM	Aceleración por GPU para el entrenamiento del modelo de aprendizaje profundo (En cumplimiento del RCD6).
Almacenamiento	SSD 500GB	Garantiza tiempos de acceso rápidos para manipulación de imágenes y modelos de IA

Nota. Elaboración propia

3.3. Inicialización

Esta fase tiene como objetivo preparar los recursos clave necesarios para el proceso de entrenamiento del modelo de aprendizaje profundo. En el presente proyecto, la fase de inicialización inicia con un análisis y preparación de los conjuntos de datos, seguido por la definición de hiperparámetros y la arquitectura del modelo. Posteriormente, se procede al entrenamiento del modelo, con el objetivo de obtener un modelo optimizado para su implementación en un entorno móvil.

Siguiendo la metodología Mobile-D, esta fase se estructura en tres etapas:

- **Configuración del Proyecto:** Preparación y análisis de los datos.
- **Planificación Inicial:** Diseño del modelo de aprendizaje profundo.
- **Desarrollo del Modelo de Deep Learning:** En este proyecto, esta fase sustituye la etapa de "Día de prueba" de Mobile-D, ya que se centra en el entrenamiento y evaluación del modelo antes de su implementación.

3.3.1. Configuración del Proyecto (Preparación y análisis de datos)

Los datos son el pilar fundamental de cualquier proyecto de aprendizaje automático (machine learning), y en el caso de la detección de neumonía mediante imágenes de rayos X, la calidad y la cantidad de datos son cruciales para el éxito del modelo. En esta sección, detallaremos los conjuntos de datos a ser utilizados y las técnicas de preprocesamiento aplicadas.

Los datos son el pilar fundamental de cualquier proyecto de aprendizaje automático (machine learning). En el caso de la detección de neumonía a partir de imágenes de rayos X, tanto la calidad como la cantidad de datos juegan un papel crucial en el rendimiento del modelo.

En esta sección se detallarán los siguientes aspectos:

- **Conjuntos de datos utilizados:** Fuente de datos, estructura y distribución de clases.
- **Técnicas de preprocesamiento:** Transformaciones aplicadas a las imágenes para mejorar la calidad del entrenamiento.

3.3.1.1. Conjuntos de datos

Para el presente proyecto se realizará el análisis de dos conjuntos de datos disponibles públicamente proporcionados por Kermany et al. [36] y la Sociedad Radiológica de América del Norte (RSNA) [40], respectivamente, siguiendo los requerimientos para el entrenamiento del modelo detallados en la Tabla 4.

- **Conjunto de datos 1:** El primer conjunto de datos descrito es el que será denominado de aquí en adelante ‘Kermany’ [36]. El mismo está disponible en los repositorios Kaggle [37] y Mendeley [39], además de ser descrito en la publicación de Cell (2018) [38]. Este conjunto de datos contiene imágenes de rayos X de tórax (anterior-posterior) de cohortes retrospectivas de pacientes pediátricos de uno a cinco años, recopiladas en el Centro Médico para Mujeres y Niños de Guangzhou, China. Todas las imágenes de rayos X de tórax se realizaron como parte de la atención clínica de rutina de los pacientes. Todas las imágenes fueron sometidas a un control de calidad para garantizar su legibilidad y fueron etiquetadas por dos médicos expertos. Un tercer experto revisó el conjunto de prueba para minimizar los posibles errores de etiquetado. [37]

Tabla 6. *Resumen del conjunto de datos 1*

	Neumonía	Normal	Total
Conjunto de entrenamiento	3883	1349	5232

Conjunto de validación	390	234	624
Total	4273	1583	5856

Nota. Elaboración propia

Para las tareas de entrenamiento es necesario realizar una división de los datos, en el presente conjunto de datos la división ya viene definida desde la fuente de la siguiente manera:

- **Conjunto de datos de Entrenamiento:** El presente conjunto de datos contiene 5.232 imágenes de rayos X para el entrenamiento del modelo, de las cuales 3.883 presentaban neumonía (2.538 bacteriana y 1.345 viral) y 1.349 eran normales.
- **Conjunto de datos de validación:** Para evaluar el rendimiento del modelo, se utilizó un conjunto de datos de prueba independiente compuesto por 624 imágenes, de las cuales 234 eran normales y 390 presentaban neumonía (242 bacteriana y 148 viral).
- **Conjunto de datos 2:** Un segundo conjunto de datos será tomado en cuenta el momento de realizar el entrenamiento. Este es provisto por el ‘Desafío de Detección de Neumonía de la RSNA (Sociedad Radiológica de Norteamérica)’ en 2018, disponible en Kaggle [40]. Este conjunto de datos es un subconjunto de aproximadamente 30000 muestras derivado del conjunto de datos NIH Chest X-ray14 [42], que a su vez es un subconjunto del conjunto de datos ChestX-ray8. Es importante destacar que las etiquetas originales del conjunto de datos NIH ChestX-ray14 provienen de informes radiológicos y se realizó el etiquetado mediante procesamiento de lenguaje natural (PLN), por lo que pueden contener imprecisiones. Para abordar esto, un aspecto fundamental en la creación de este

conjunto de datos fue el riguroso proceso de anotación y adjudicación realizado por radiólogos expertos. Seis radiólogos certificados anotaron las 30.000 radiografías de tórax, identificando opacidades pulmonares sospechosas de neumonía y delimitando su ubicación mediante cuadros delimitadores (bounding boxes). Este proceso minucioso permitió obtener un conjunto de datos de alta calidad para el entrenamiento y la evaluación del modelo de detección de neumonía [42].

A continuación, se detallan las características de este segundo conjunto de datos:

- **Características Poblacionales:** Un aspecto clave en la detección de neumonía a partir de imágenes de rayos X es la distribución etaria de los pacientes incluidos en el conjunto de datos. La edad del paciente puede influir significativamente en la aparición y presentación radiológica de la neumonía, por lo que garantizar una diversidad etaria adecuada es crucial para el entrenamiento del modelo. Dado que este conjunto de datos está compuesto por archivos en formato DICOM, es posible extraer los metadatos asociados y analizar la distribución etaria mediante técnicas de Análisis Exploratorio de Datos (EDA, por sus siglas en inglés). A través de este análisis, se obtiene una distribución detallada de las edades de los pacientes en diferentes rangos etarios (véase la Tabla 7) además de una distribución acumulativa de exámenes en función de la edad (Véase la Figura 21), lo que permite evaluar la representatividad de cada grupo poblacional en el proceso de entrenamiento del modelo.

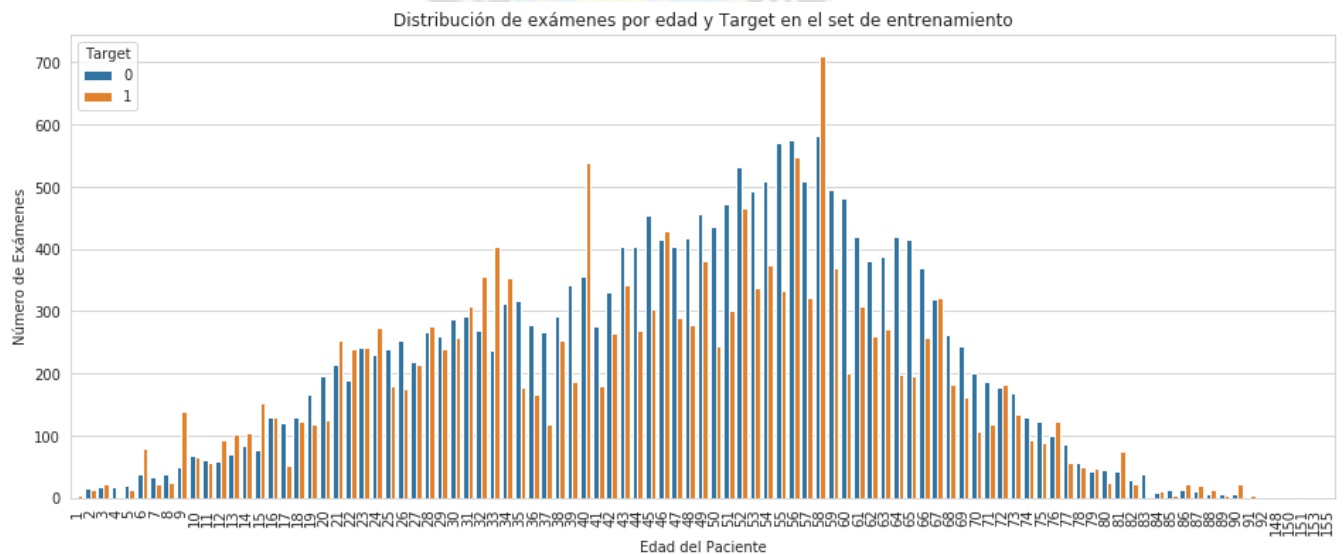
Tabla 7. *Distribución de edades por rangos*

Rango de edad	Número de Pacientes
0-10 años	435 pacientes

10-20 años	1468 pacientes
20-30 años	3205 pacientes
30-40 años	3936 pacientes
40-50 años	5048 pacientes
50-60 años	6619 pacientes
60-70 años	4279 pacientes
70-80 años	1452 pacientes
80-90 años	233 pacientes
90-100 años	4 pacientes

Nota. Elaboración propia

Figura 21. *Distribución de exámenes por edad y Clase en el conjunto de datos 2*



Nota. Elaboración propia

El análisis de la distribución etaria en el conjunto de datos permite identificar los grupos de pacientes mejor representados en el entrenamiento del modelo. Con base en este análisis, se ha determinado que la mayoría de los exámenes de rayos X provienen de pacientes entre los 30 y 70 años (Mayor a 3000 muestras en esos rangos), lo que sugiere que el modelo que se entrene con este conjunto de datos estará mejor optimizado para su uso en adultos y adultos mayores.

- **Descripción detallada del conjunto de datos:** El conjunto de datos contiene: 26684 imágenes en formato DICOM para las etapas de entrenamiento y evaluación y 3000 imágenes en formato DICOM sin etiquetar para una evaluación posterior.

Adicionalmente a las imágenes, las etiquetas son definidas mediante dos archivos CSV:

- **stage_2_detailed_class_info.csv:** archivo de valores separados por comas (CSV) con dos columnas:

- **patientId:** Un identificador único para cada paciente en el conjunto de datos.
- **class:** La etiqueta asignada a la radiografía de tórax de cada paciente.

Se definen mediante tres clases:

- ‘No Lung Opacity / Not Normal’
- ‘Normal’
- ‘Lung Opacity’

En la tabla 8 se puede observar una muestra de los primeros 5 registros.

Tabla 8. *Muestra de los primeros 5 registros del archivo stage_2_detailed_class_info.csv*

	patientId	class
0	0004cfab-14fd-4e49-80ba-63a80b6bddd6	No Lung Opacity / Not Normal
1	00313ee0-9eaa-42f4-b0ab-c148ed3241cd	No Lung Opacity / Not Normal
2	00322d4d-1c29-4943-afc9-b6754be640eb	No Lung Opacity / Not Normal
3	003d8fa0-6bf1-40ed-b54c-ac657f8495c5	Normal
4	00436515-870c-4b36-a041-de91049b9ab4	Lung Opacity

Nota. Elaboración propia

Acerca del etiquetado de las clases, en [42] se puede obtener un detalle más preciso de las mismas:

- **Lung Opacity (Alta probabilidad):** Opacidad pulmonar con alta probabilidad de ser neumonía. Esto incluye áreas que cumplen con la definición de Fleischner (más opacas que el área circundante) y no corresponden a las exclusiones (masas, nódulos, colapsos, atelectasias). Esta clase es la principal para el entrenamiento de detección de neumonía.
 - **No Opacity / Not Normal:** Radiografías que no muestran opacidad sospechosa de neumonía, pero que pueden presentar otras anomalías sutiles o hallazgos que requieren una interpretación más detallada por un radiólogo.
 - **Normal:** Radiografías sin opacidad pulmonar ni ninguna otra anomalía visible.
- **stage_2_train_labels.csv:** archivo de valores separados por comas (CSV) con seis columnas:
- **patientId:** Un identificador único para cada paciente en el conjunto de datos.
 - **x, y:** Coordenadas (x, y) del punto superior izquierdo de un cuadro delimitador (bounding box) que encierra una opacidad pulmonar detectada en la radiografía
 - **width, height:** Ancho y altura del cuadro delimitador en píxeles.

- **Target:** Con valor igual a 1 si hay una alta probabilidad de ser neumonía y tienen bounding boxes definidos y con valor igual a 0 si no tiene ningún bounding box. Coincide con la clase ‘Lung Opacity’ del anterior archivo CSV.

En la tabla 9 se observa una muestra de los primeros 5 registros.

Tabla 9. *Muestra de los primeros 5 registros del archivo stage_2_train_labels.csv*

	patientId	x	y	width	height	Target
0	0004cfab-14fd-4e49-80ba-63a80b6bddd6	NaN	NaN	NaN	NaN	0
1	00313ee0-9eaa-42f4-b0ab-c148ed3241cd	NaN	NaN	NaN	NaN	0
2	00322d4d-1c29-4943-afc9-b6754be640eb	NaN	NaN	NaN	NaN	0
3	003d8fa0-6bfl-40ed-b54c-ac657f8495c5	NaN	NaN	NaN	NaN	0
4	00436515-870c-4b36-a041-de91049b9ab4	264.0	152.0	213.0	379.0	1

Nota. Elaboración propia

Tomando en cuenta que el presente conjunto de datos también puede ser utilizado para la tarea de Detección de Objetos (Object Detection), el mismo tiene información de ‘Bounding Boxes’ (una fila por cada bounding box en los archivos CSV) es por ello que al existir en algunos casos más de un bounding box por imagen hay filas con id repetidos, dando un total de 30227 filas en ambos archivos CSV. Al no ser información relevante para el presente proyecto se obtiene el número de pacientes únicos que coincide con el número de muestras provistas por el conjunto de datos: 26684 obteniendo el detalle por archivo CSV en las tablas 10 y 11.

Tabla 10. *Resumen stage_2_detailed_class_info.csv*

	Lung Opacity	Normal	No Lung Opacity / Not Normal	Total
Train	6012	8851	11821	26684

Nota. Elaboración propia

Tabla 11. *Resumen stage_2_train_labels.csv*

	Target=1	Target=0	Total
Train	6012	20672	26684

Nota. Elaboración propia

Tomando en cuenta la información provista por el archivo ‘stage_2_detailed_class_info.csv’, tomaremos la siguiente estrategia de división del conjunto de datos: un porcentaje de los datos (80%) para el entrenamiento, otro porcentaje (10%) para la validación del modelo y otro porcentaje (10%) para las pruebas que se realizarán en el siguiente capítulo [44] como se muestra en la tabla 12.

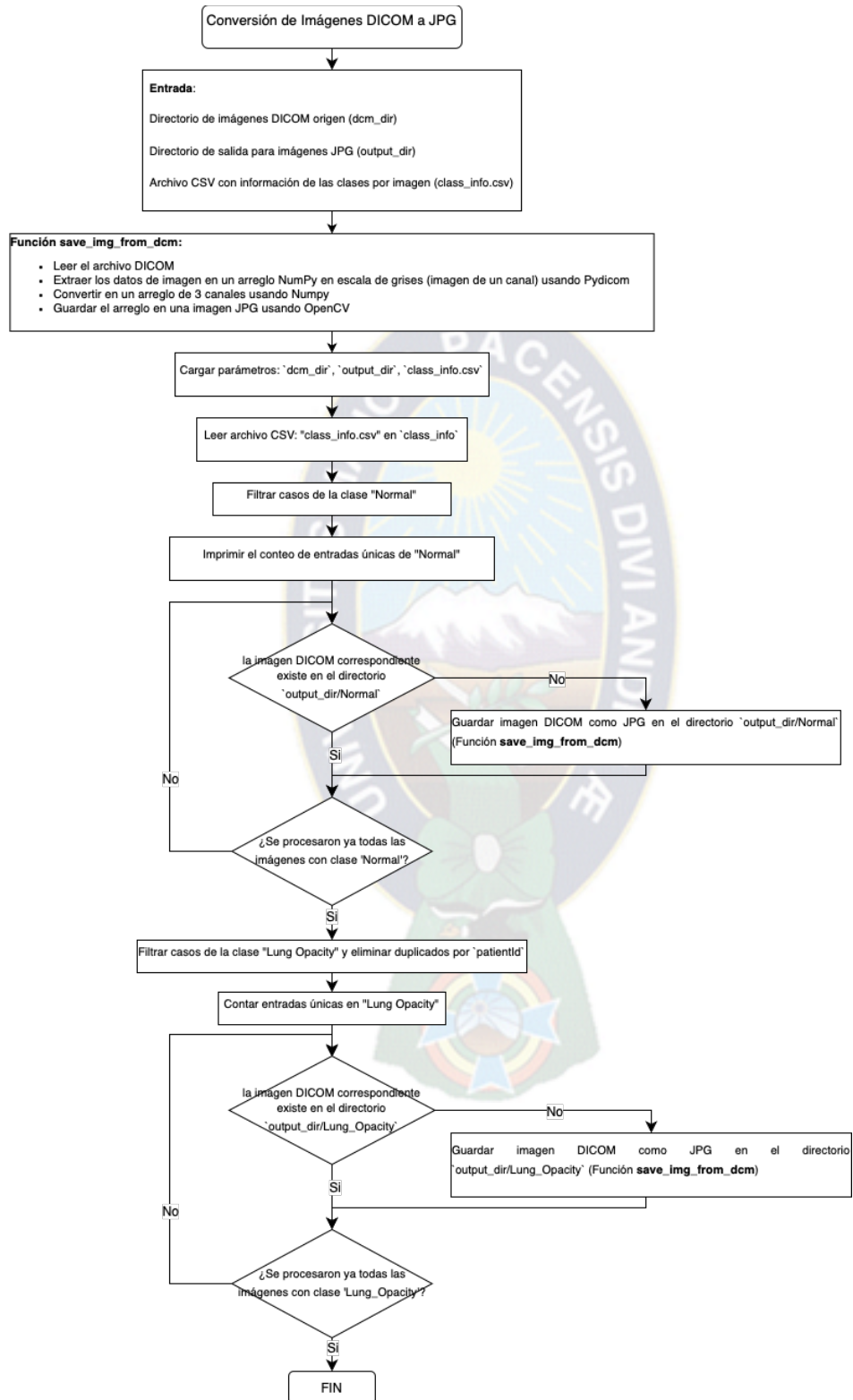
Tabla 12. *Resumen del conjunto de datos 2*

	Lung Opacity	Normal	Total
Conjunto de entrenamiento	4808	7079	11887
Conjunto de validación	602	886	1488
Conjunto de pruebas	602	886	1488
Total	6012	8851	14863

Nota. Elaboración propia

En el conjunto de datos 2 es necesario una etapa de conversión de las imágenes previas a la presentación del conjunto de datos al entrenamiento del modelo, esto debido a que inicialmente se encuentran en formato DICOM (.dcm), el mismo que no es soportado por las librerías y frameworks que utilizaremos. Para ello se realizará una conversión de DICOM a JPG. En la figura 22 se muestra el algoritmo de conversión.

Figura 22. *Algoritmo de conversión de imágenes DICOM a JPG*



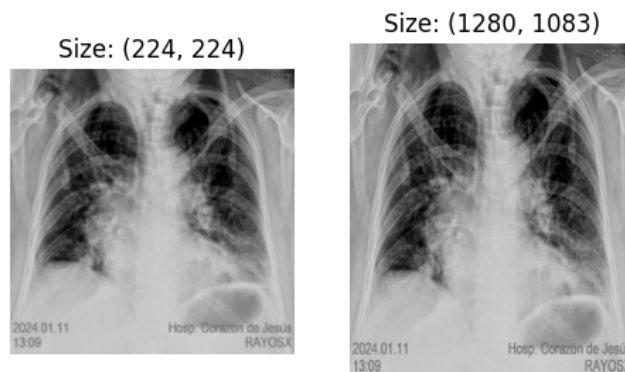
Nota. Elaboración propia

3.3.1.2. Preprocesamiento de Datos

Una vez descritos los conjuntos de datos que serán utilizados, se describirá el preprocesamiento de imágenes realizado, puesto que este es un paso crucial para preparar los datos de entrada para el modelo de aprendizaje automático. En este proyecto, se aplicarán diversas transformaciones a las imágenes de rayos X, utilizando clases propias de Torchvision y el framework de aprendizaje profundo Pytorch, con el objetivo de mejorar la capacidad de generalización del modelo, aumentar la diversidad de los datos de entrenamiento y estandarizar el formato de las imágenes.

- **Transformaciones para el Conjunto de Entrenamiento (Train):** Las imágenes en el conjunto de entrenamiento se someterán a las siguientes transformaciones:
 - **transforms.Resize((224, 224)):** Todas las imágenes se redimensionaron a un tamaño uniforme de 224x224 píxeles. Esta estandarización es necesaria para que el modelo pueda procesar imágenes de diferentes tamaños originales. También es necesaria debido a que se utilizará un modelo con pesos pre-entrenados en el conjunto de datos ImageNet, este modelo requiere esas dimensiones como entrada.

Figura 23. *Ejemplo de redimensionamiento de una imagen radiológica*

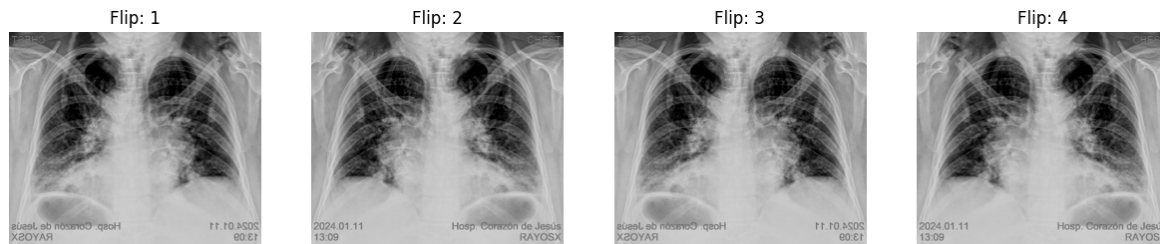


Nota. Elaboración propia

- **transforms.RandomHorizontalFlip:** Se aplicó un volteo horizontal aleatorio a algunas imágenes. Esto introduce variaciones en los datos de entrenamiento,

ayudando al modelo a aprender características relevantes independientemente de la orientación horizontal de la imagen.

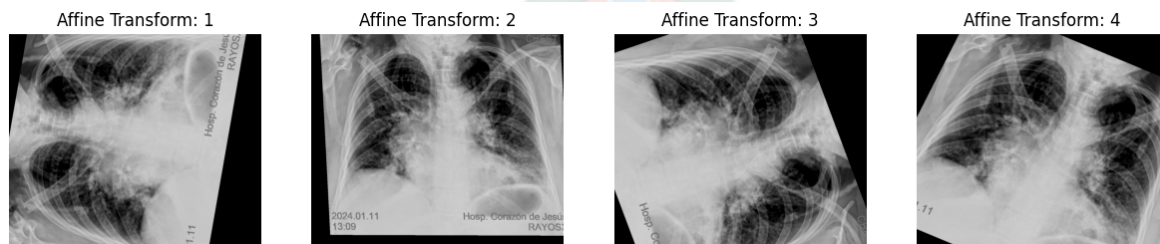
Figura 24. *Ejemplo de volteo horizontal aleatorio*



Nota. Elaboración propia

- **transforms.RandomVerticalFlip:** Similar al volteo horizontal, se aplicó un volteo vertical aleatorio para aumentar aún más la diversidad de los datos.
- **transforms.RandomAffine:** Se realizaron transformaciones afines aleatorias, incluyendo rotaciones (entre -180° y 180°), traslaciones (desplazamientos de hasta el 10% en cada eje), escalado (entre 0.9 y 1.1) y cizallamiento (con un factor de hasta 5). Estas transformaciones simulan variaciones en la posición, tamaño y forma de los pulmones en las imágenes, lo que ayuda al modelo a ser más robusto ante estas variaciones en datos reales.

Figura 25. *Ejemplo de transformaciones afines aleatorias*



Nota. Elaboración propia

- **transforms.ToTensor():** Las imágenes se convirtieron al formato de tensor de PyTorch, el cual es el formato requerido para el entrenamiento del modelo.

- **transforms.Normalize(mean, std):** Realiza una normalización de los valores de los píxeles de la imagen. Esta normalización se aplica a cada canal de la imagen (rojo, verde y azul) de forma independiente. La fórmula de normalización resultante de transforms.ToTensor() y transforms.Normalize(mean, std) es la siguiente:

$$\text{output[channel]} = \frac{\frac{\text{input[channel]}}{255} - \text{mean[channel]}}{\text{std[channel]}} \quad (5)$$

Donde:

- input[channel] es el valor original del píxel en el canal.
- output[channel] es el valor normalizado del píxel en el canal.
- mean[channel] es la media del canal en el conjunto de datos ImageNet.
- std[channel] es la desviación estándar del canal en el conjunto de datos ImageNet.

En este caso, se utilizan los siguientes valores para la media y la desviación estándar:

- mean = [0.485, 0.456, 0.406]
- std = [0.229, 0.224, 0.225]

Estos valores se obtuvieron a partir del conjunto de datos ImageNet (Véase la sección 2.1.3.3.1), que es un conjunto de datos muy grande y diverso que se utiliza comúnmente para entrenar modelos de visión artificial. Al utilizar la media y la desviación estándar de ImageNet, se asegura que la normalización sea consistente con la que se utilizó para entrenar el modelo ResNet-50 pre-entrenado (Véase la sección 2.1.3.4.1).

- **Transformaciones para el Conjunto de Validación (Val):** Las imágenes en el conjunto de validación se sometieron a un preprocesamiento más simple:
 - **transforms.Resize((224, 224)):** Redimensionamiento a 224x224 píxeles, igual que en el conjunto de entrenamiento.
 - **transforms.ToTensor():** Conversión al formato de tensor de PyTorch.

- **transforms.Normalize(mean, std):** Normalización de los valores de los píxeles, utilizando la misma media y desviación estándar que en el conjunto de entrenamiento.

Es importante destacar que no se aplicaron transformaciones de volteo o afines al conjunto de validación, ya que el objetivo de este conjunto es evaluar el rendimiento del modelo en datos sin alteraciones, simulando un escenario real.

3.3.2. Planificación Inicial

Luego de la definición de los conjuntos de datos en la etapa de “Configuración del Proyecto”, se procede a diseñar el modelo de aprendizaje profundo que será utilizado en el presente proyecto. En esta sección se presenta la selección de la arquitectura del modelo, su justificación técnica y las modificaciones realizadas para adaptar la red neuronal a la tarea de clasificación de neumonía en imágenes de rayos X.

3.3.2.1. Arquitectura del Modelo (ResNet50)

Las arquitecturas de redes neuronales convolucionales (CNN) están diseñadas para abordar problemas de clasificación de imágenes mediante la extracción jerárquica de características. En este proyecto se compararon diferentes modelos preentrenados en ImageNet para seleccionar la arquitectura más adecuada para la tarea de detección de neumonía.

3.3.2.1.1. Comparación de Modelos Preentrenados

Se evaluaron las siguientes arquitecturas preentrenadas en el contexto de detección de neumonía en imágenes de rayos X.

Tabla 13. Comparación de arquitecturas

Modelo	Profundidad	Principales Características
VGG-16	16 capas	Filtros 3x3, arquitectura profunda estándar.

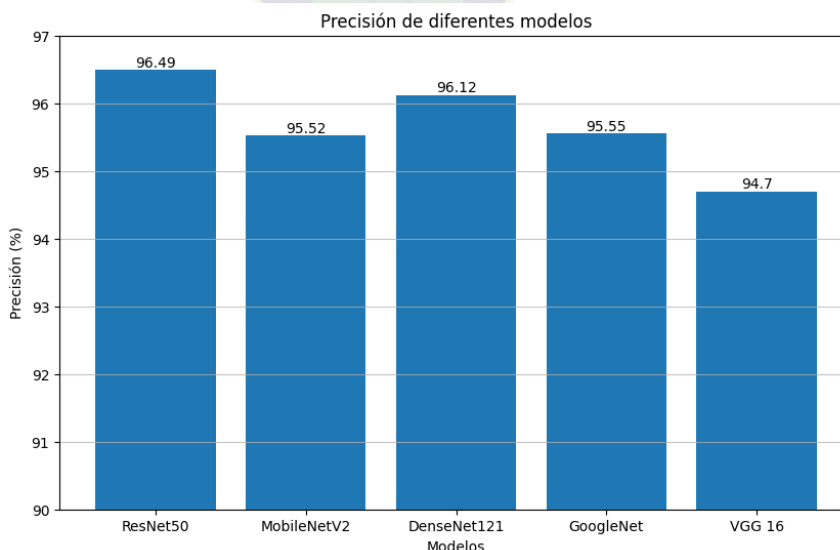
GoogleNet	22 capas	Módulos <i>Inception</i> con convoluciones paralelas.
DenseNet-121	121 capas	Conexiones densas entre capas, mejora el gradiente.
MobileNetV2	53 capas	Convoluciones separables, eficiente en móviles.
ResNet-50	50 capas	Bloques residuales, evita la desaparición del gradiente.

Nota. Elaboración propia

3.3.2.1.2. Selección de la Arquitectura

De acuerdo con la investigación de Pramanik et al. [34], ResNet50 destaca por su rendimiento superior en la detección de neumonía. Esta arquitectura utiliza bloques residuales, que facilitan el entrenamiento de redes profundas mediante la mitigación del problema de la desaparición del gradiente. Los resultados mostrados en la Figura 26 evidencian la superioridad de ResNet-50 sobre otros modelos en el primer pliegue del experimento de Pramanik, confirmando su capacidad para extraer características relevantes y mejorar la precisión de la clasificación, cumpliendo además con el requerimiento RNF1 relacionado a la precisión del modelo. Esta arquitectura será la utilizada en el presente proyecto.

Figura 26. Rendimiento de clasificación de varias arquitecturas



Nota. Elaboración propia basada en Pramanik, R., Sarkar, S., & Sarkar, R. (2022). An adaptive and altruistic PSO-based deep feature selection method for Pneumonia detection from Chest X-rays. Applied Soft Computing, 128, 109464.[34]

3.3.2.2. Adaptación del Modelo

Con el objetivo de adaptar ResNet-50 a la tarea específica de detección de neumonía y al requerimiento funcional RF3 relacionado a realizar una clasificación binaria, se han realizado modificaciones en sus capas finales utilizando conceptos de transferencia de aprendizaje.

3.3.2.2.1. Proceso de adaptación del modelo

El proceso de adaptación del modelo ResNet-50 se estructuró en cuatro pasos clave, los cuales permitieron ajustar la arquitectura original del modelo para la tarea específica de detección de neumonía.

Tabla 14. *Resumen del proceso de adaptación de modelo*

Paso	Descripción
1. Carga del modelo preentrenado	Se descarga la arquitectura ResNet-50 con pesos preentrenados en <i>ImageNet</i> .
2. Transferencia de aprendizaje	Se conservan las capas convolucionales iniciales para reutilizar la capacidad de extracción de características.
3. Eliminación de la capa de clasificación	La capa final de 1000 clases se reemplaza por una arquitectura personalizada de clasificación binaria.
4. Adición de capas personalizadas	Se agregan capas lineales y activaciones para mejorar la adaptación a la nueva tarea.

Nota. Elaboración propia

- **Carga y Preparación del Modelo Base:** Se utilizó la arquitectura preentrenada de ResNet-50 como base del modelo. Este modelo, originalmente diseñado para clasificar 1000 categorías en ImageNet, es ideal para la extracción de características debido a su profundidad y capacidad de representación.
 - Se descargaron los pesos preentrenados de ResNet-50.

- El modelo se trasladó a GPU para aprovechar la aceleración por hardware y mejorar la eficiencia del entrenamiento.
- **Transferencia de Aprendizaje:** La transferencia de aprendizaje permite reutilizar las capas convolucionales iniciales de ResNet-50, que ya han aprendido a identificar patrones visuales genéricos. Estas capas son fundamentales para la extracción de características relevantes en imágenes de rayos X.
 - Se conservaron las capas convolucionales y únicamente se modificó la capa de clasificación final.
 - Esto permite que el modelo aproveche el conocimiento previo en la identificación de patrones y reduzca la necesidad de entrenar desde cero.
- **Eliminación de la Capa de Clasificación Original:** Dado que las capas finales de ResNet-50 están diseñadas para clasificar 1000 categorías, se eliminó la capa de clasificación original para ajustarla a la tarea binaria (Neumonía / Normal).
 - Se utilizaron herramientas de PyTorch (nn.Sequential) para acceder a todas las capas del modelo excepto la última.
 - Se mantuvieron intactas las capas iniciales, ya que son responsables de la extracción de características de bajo y alto nivel.
- **Adición de Capas Personalizadas:** Para adaptar ResNet-50 a la clasificación binaria, se agregaron capas lineales adicionales con una función de activación no lineal LeakyReLU.

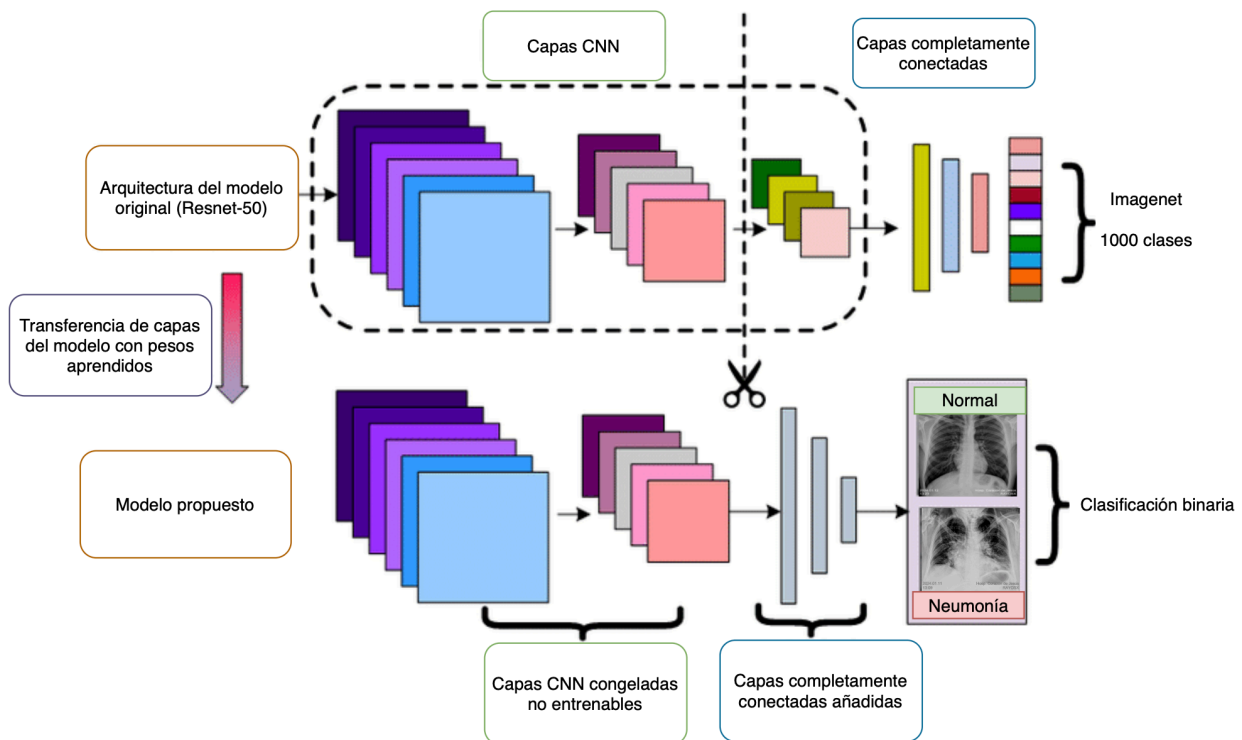
3.3.2.2.2. *Diseño de la Nueva Capa de Clasificación*

La arquitectura original de ResNet-50 finaliza con una capa lineal de 1000 neuronas diseñada para clasificar imágenes en 1000 categorías de ImageNet. Para la tarea de detección de

neumonía siguiendo el RF3, se reemplazó esta capa por una estructura más compacta, adaptada a la clasificación binaria (Tiene Neumonía / No tiene neumonía).

- La primera capa lineal (Linear(2048, 512)) reduce la dimensionalidad de las características extraídas por ResNet-50, facilitando la representación del espacio de características.
- La función de activación LeakyReLU introduce no linealidad en la red, permitiendo una mejor propagación del gradiente en valores cercanos a cero, lo que evita el problema de activaciones muertas que puede ocurrir con la función ReLU estándar.
- Finalmente, la segunda capa lineal (Linear(512, 2)) genera la salida final del modelo, clasificando la imagen en una de las dos categorías de interés: neumonía o normal.

Figura 27. Diagrama del proceso de transferencia de aprendizaje aplicado



Nota. Elaboración propia adaptada de Rahman, M. R., Tabassum, S., Haque, E., Nishat, M. M., Faisal, F., & Hossain, E. (2021, December). CNN-based deep learning approach for micro-crack detection of solar panels. In *2021 3rd International Conference on Sustainable Technologies for Industry 4.0 (STI)* (pp. 1-6). IEEE. [70]

Tabla 15. *Modificaciones en la arquitectura de ResNet-50*

Capa Original	Capa Modificada	Descripción
fc (1000)	Linear(2048, 512)	Reduce la dimensionalidad de la representación.
—	LeakyReLU()	Introduce no linealidad, previene activaciones muertas.
—	Linear(512, 2)	Genera la salida binaria (Neumonía / Normal).

Nota. Elaboración propia

3.3.2.2.3. Implementación Técnica

La implementación de estas modificaciones se realizó en el framework PyTorch, encapsulando la arquitectura final en una clase llamada ConvNet, la cual integra la base convolucional de ResNet-50 con las nuevas capas de clasificación.

Figura 28. *Fragmento de código aplicando transferencia de aprendizaje*

```
class ConvNet(nn.Module):
    def __init__(self, model, num_classes):
        super(ConvNet, self).__init__()
        self.base_model = nn.Sequential(*list(model.children())[:-1])
        self.linear1 = nn.Linear(in_features=2048, out_features=512)
        self.linear2 = nn.Linear(in_features=512, out_features=num_classes)
        self.relu = nn.LeakyReLU()

    def forward(self, x):
        x = self.base_model(x)
        x = torch.flatten(x, 1)
        x = self.linear1(x)
        lin = self.relu(x)
        x = self.linear2(lin)
        return lin, x
```

Nota. Elaboración propia

Finalmente, el modelo modificado fue instanciado y configurado para ejecutarse en GPU, asegurando un entrenamiento eficiente y una correcta adaptación a la tarea de detección de neumonía.

3.3.3. Desarrollo del Modelo de Deep Learning

Con la información recopilada en las etapas anteriores, que incluyen la selección de conjuntos de datos, la arquitectura del modelo y su adaptación a la tarea específica, se procede con la configuración del entrenamiento, el proceso de ajuste de pesos y la evaluación del modelo. El desarrollo del modelo sigue la estrategia de aprendizaje por transferencia, utilizando ResNet-50 preentrenado y ajustándolo a los datos específicos de la tarea con un conjunto de hiperparámetros optimizados. Finalmente, el modelo se prepara para su implementación en dispositivos móviles.

3.3.3.1. Configuración del Entrenamiento

La configuración del entrenamiento es crucial para obtener buenos resultados en el modelo de detección de neumonía. Para lograr un entrenamiento eficiente y reducir el riesgo de sobreajuste, se seleccionaron cuidadosamente los hiperparámetros tomando como referencia las investigaciones [34] y [14]. En [34] son definidos mediante el uso del método de búsqueda en cuadrícula (*grid search*), pudiendo observarse los resultados en la tabla 16.

Tabla 16. Comparación de diferentes tasas de aprendizaje y tamaños de lote

Tasa de aprendizaje	Tamaño de lote	Precisión (en %)
1e-3	16	94.32
	32	96.32
	64	95.47
1e-4	16	96.12
	32	96.49
	64	95.38
1e-5	16	93.07
	32	92.74
	64	91.62

Nota. Elaboración propia basa de en la comparación de diferentes tasas de aprendizaje y tamaños de lote en Fold-1 en el “Conjunto de datos 1” descrito en Pramanik, R., Sarkar, S., & Sarkar, R. (2022). An adaptive and altruistic PSO-based deep feature selection method for Pneumonia detection from Chest X-rays. Applied Soft Computing, 128, 109464. [34]

3.3.3.1.1. *Hiperparámetros Seleccionados*

A continuación, se detallan los principales hiperparámetros, la función de pérdida y el método de optimización a ser utilizados:

- **Tamaño del Lote (Batch Size):** Durante los experimentos descritos en las investigaciones, se consideraron los tamaños de lote de {16, 32, 64} para encontrar el valor óptimo. Se observó que un tamaño de lote de **32** proporcionaba el mejor rendimiento para la tarea de clasificación binaria entre neumonía y normal.
- **Tasa de Aprendizaje (Learning Rate):** Se probaron diferentes valores de tasa de aprendizaje { $1e-3$, $1e-4$, $1e-5$ } y se determinó que un valor de **$1e-4$** era óptimo. Además, se implementó un ajuste dinámico de la tasa de aprendizaje (learning rate scheduler), reduciéndola en un factor de diez cada 5 épocas para suavizar el proceso de aprendizaje y mitigar el riesgo de sobreajuste.
- **Función de Pérdida:** Se utilizará la función de pérdida de entropía cruzada (CrossEntropyLoss). Esta función es ampliamente utilizada en problemas de clasificación, como la detección de neumonía, donde el objetivo es asignar una probabilidad a cada clase posible.
- **Optimizador:** El algoritmo de optimización juega un papel fundamental en el proceso de entrenamiento de una red neuronal profunda, ya que de él depende el rendimiento y la eficiencia del aprendizaje. Se requiere un algoritmo que garantice la convergencia del modelo hacia la minimización del error, al tiempo que mantiene tiempos de entrenamiento razonables. En este proyecto, se seleccionó el algoritmo Adam (Adaptive Moment Estimation, algoritmo de optimización que adapta la tasa de aprendizaje de cada parámetro individualmente en función de su estimación del momento y de la magnitud del gradiente) debido a su capacidad para asegurar la convergencia de la red mediante el uso de momentos

de primer y segundo orden. Adam es un algoritmo ampliamente recomendado y eficaz para el entrenamiento de redes neuronales profundas en tareas de clasificación, como la que se aborda en este trabajo.

- **Dispositivo de Cómputo:** Para optimizar el tiempo de entrenamiento, el modelo se ejecutará en una GPU (Se ejecutará sobre una GPU NVIDIA GeForce RTX 2060 6GBytes VRAM). De todas formas el código implementa una verificación (`torch.cuda.is_available()`) que determina automáticamente la disponibilidad de una GPU. En caso de no contar con una GPU, el entrenamiento se realizará en la CPU.

3.3.3.1.2. Configuración de DataLoaders

Se crearon dos dataloaders, uno para el conjunto de entrenamiento y otro para el conjunto de validación, utilizando la clase `DataLoader`. Los dataloaders permiten cargar los datos en lotes durante el entrenamiento. Se configuraron los siguientes parámetros:

Tabla 17. Configuración de DataLoaders

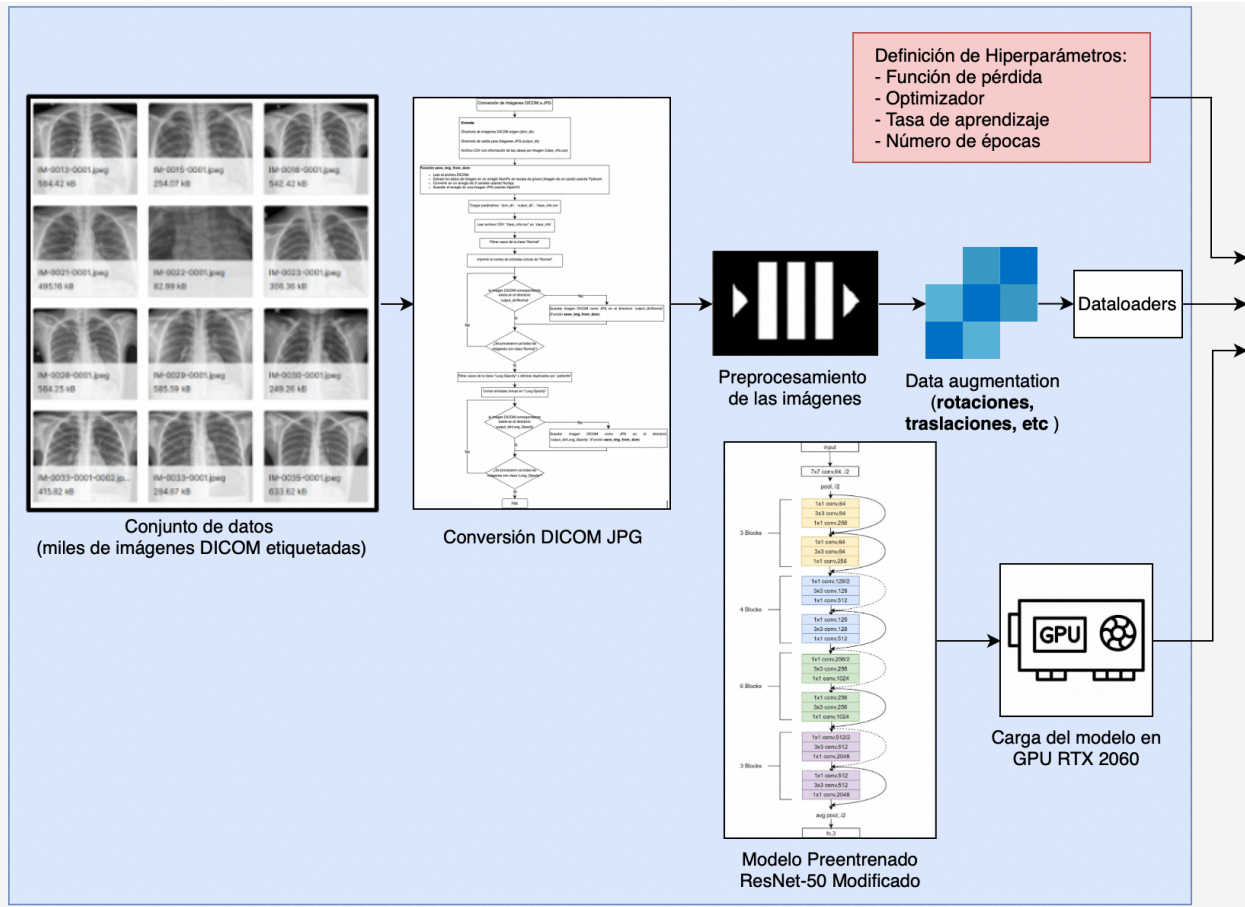
Parámetro	Entrenamiento	Validación
batch_size	32	1
shuffle	True (Aleatorización de datos)	False (No aleatorizado)
num_workers	4	4

Nota. Elaboración propia

- **Conjunto de entrenamiento:** Como se detalló antes, el tamaño del lote será de 32 (`batch_size=32`), otros parámetros a ser configurados: `shuffle=True`, `num_workers=4`. El parámetro `shuffle=True` asegura que los datos se mezclen aleatoriamente en cada época, lo que ayuda a prevenir el sobreajuste. El parámetro `num_workers=4` permite cargar los datos en paralelo utilizando múltiples subprocesos, lo que acelera el proceso de entrenamiento.

- **Conjunto de validación:** Para el conjunto de validación los parámetros serán: batch_size=1, shuffle=False, num_workers=4. En la fase de validación, no es necesario mezclar los datos (shuffle=False), ni tampoco definir el mismo tamaño de lote que al realizar el entrenamiento.

Figura 29. Módulo de Preparación para el Entrenamiento



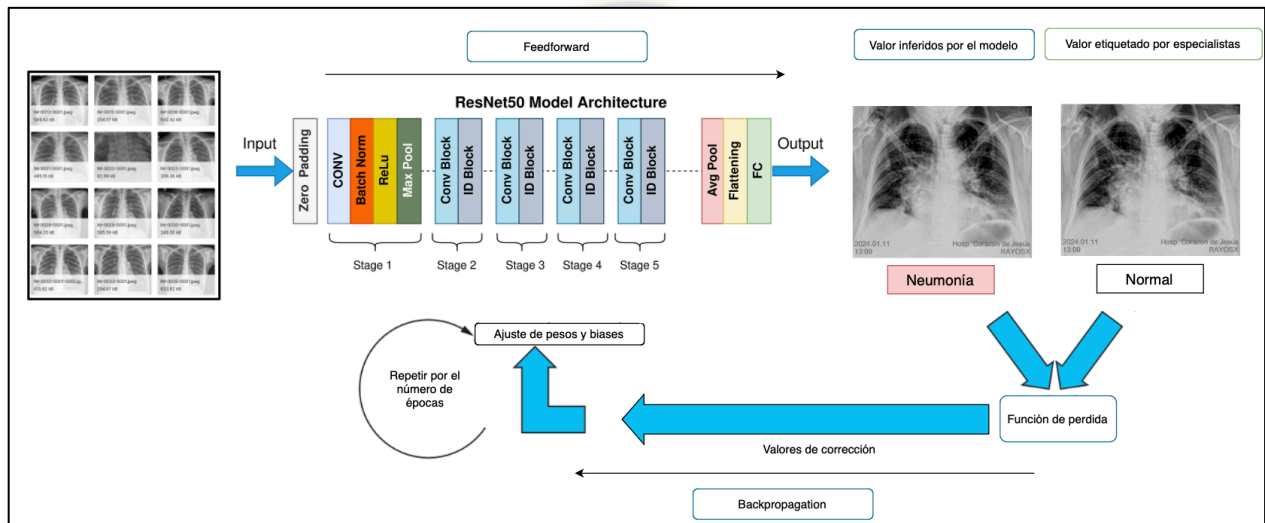
Nota. Elaboración propia. Los componentes de este módulo son descritos en las secciones 3.3.1, 3.3.2, y en la sección 3.3.3.

Antes de describir el proceso de entrenamiento en la Figura 29 podemos observar todos los componentes hasta hora descritos que en su conjunto forman parte de un módulo de preparación para el proceso de entrenamiento

3.3.3.2. Proceso de Entrenamiento y Evaluación

Una vez definida la arquitectura y los hiperparámetros, se procede con el entrenamiento del modelo. Se utiliza un enfoque basado en dos conjuntos de datos (Kermany y RSNA) y diferentes valores de épocas para evaluar el rendimiento. Los resultados se expondrán en el siguiente capítulo con detalle (Véase Capítulo 4 Análisis y discusión de resultados).

Figura 30. Esquema inicial del entrenamiento del modelo



Nota. Elaboración propia basada en Dressen, O. (n.d.). *Training Convolutional Neural Networks: What Is Machine Learning? Part 2*. <https://www.analog.com/en/resources/analog-dialogue/articles/training-convolutional-neural-networks-what-is-machine-learning-part-2.html> [71]

3.3.3.2.1. Implementación del Entrenamiento

El entrenamiento del modelo se implementa utilizando el framework PyTorch en Python. Se define una función `train_model` (Véase el Anexo A.1) que realiza el proceso de entrenamiento. Esta función sintetiza el flujo que describe la Figura 31. Esta función recibe como argumentos:

- El modelo
- La función de pérdida
- El optimizador
- La tasa de aprendizaje
- Los dataloaders (para los conjuntos de entrenamiento y validación)